

# 第 1 卷 C++17 / STL / 输入输出

考试速查：主查 C++ 现场写法：标准头文件、输入输出、保留小数、右对齐、EOF、string 成员函数、STL 容器和常用算法。

## Contents

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>第 1 卷 C++17 / STL / 输入输出</b>                       | <b>3</b>  |
| 考试速查定位                                                | 3         |
| 头文件安全包                                                | 3         |
| 输入输出索引表                                               | 3         |
| STL 函数索引表                                             | 3         |
| 模块目录                                                  | 3         |
| <b>CPP-001 C++17 主骨架与输入输出</b>                         | <b>4</b>  |
| <b>CPP-10: 输入输出与格式化</b>                               | <b>6</b>  |
| 考场目标                                                  | 7         |
| 1. 先选 IO 套路                                           | 7         |
| 1A. 分隔方式总表                                            | 8         |
| 空格和换行等价的例子                                            | 8         |
| 行边界有意义的例子                                             | 8         |
| 2. cin/cout 常用片段                                      | 9         |
| 快速 cin/cout 骨架                                        | 9         |
| 多组数据                                                  | 9         |
| EOF 读到输入末尾就停止                                         | 9         |
| 小数保留位数                                                | 10        |
| 右对齐、左对齐、补零                                            | 10        |
| 3. scanf/printf 常用片段                                  | 11        |
| 基础骨架                                                  | 11        |
| 常用格式符                                                 | 11        |
| 小数、宽度、补零                                              | 11        |
| 4. 自写快读快写                                             | 11        |
| 5. 字符、整行与换行处理                                         | 13        |
| 读一个非空白字符                                              | 13        |
| 读下一个原始字符                                              | 13        |
| 读整行                                                   | 13        |
| 6. 复杂格式处理                                             | 14        |
| 标点固定：用 char 接住                                        | 14        |
| 一行里有不定个整数                                             | 14        |
| 简单逗号分隔                                                | 14        |
| key=value 或 key: value                                | 14        |
| 读带空格的名字和值                                             | 15        |
| 网格读入                                                  | 15        |
| 7. 提交前格式检查                                            | 15        |
| <b>CPP-002 vector/string/pair/tuple 基础容器</b>          | <b>15</b> |
| <b>CPP-011 string 考场速查</b>                            | <b>17</b> |
| 1. 构造、长度、访问                                           | 18        |
| 2. 修改：push_back / pop_back / insert / erase / replace | 19        |
| 3. substr / find / rfind                              | 19        |
| 4. compare 与字典序                                       | 20        |
| 5. getline 混 cin                                      | 20        |
| 6. stoi / stoll / to_string                           | 21        |
| 7. 字符函数：isdigit / isalpha / tolower / toupper         | 22        |
| 8. 按字符计数                                              | 22        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| 9. split 思路 . . . . .                                   | 23        |
| 10. 可抄完整模板 . . . . .                                    | 23        |
| 11. 常见坑 . . . . .                                       | 24        |
| <b>STR-01 string 常用操作</b>                               | <b>25</b> |
| <b>CPP-013 STL 容器成员函数速查</b>                             | <b>27</b> |
| 顺序容器简表 . . . . .                                        | 28        |
| 通用成员函数速查 . . . . .                                      | 28        |
| vector/deque/list/array/string 常用模板 . . . . .           | 29        |
| stack/queue/priority_queue . . . . .                    | 29        |
| priority_queue 存 pair 与自定义排序 . . . . .                  | 30        |
| set/multiset . . . . .                                  | 31        |
| map/multimap . . . . .                                  | 31        |
| unordered_map/unordered_set . . . . .                   | 32        |
| pair/tuple . . . . .                                    | 32        |
| 迭代器遍历与 erase 安全写法 . . . . .                             | 32        |
| 常用完整模板 . . . . .                                        | 33        |
| <b>CPP-012 STL 常用算法速查</b>                               | <b>35</b> |
| 1. 考场总表 . . . . .                                       | 35        |
| 2. 排序、比较函数与 lambda 正确写法 . . . . .                       | 36        |
| 3. 半开区间与闭区间换算 . . . . .                                 | 37        |
| 4. 去重与删除 . . . . .                                      | 37        |
| 5. 二分查找与区间计数 . . . . .                                  | 37        |
| 6. 最大最小、第 k 小与重排 . . . . .                              | 38        |
| 7. 排列枚举 . . . . .                                       | 39        |
| 8. 批量赋值、编号、求和与前缀和 . . . . .                             | 39        |
| 9. gcd 与 lcm . . . . .                                  | 39        |
| 10. 统计、查找与判定 . . . . .                                  | 40        |
| 11. 综合模板 . . . . .                                      | 40        |
| <b>CPP-003 sort/lambda/比较函数/lower_bound/upper_bound</b> | <b>42</b> |
| <b>CPP-004 queue/deque/stack/priority_queue</b>         | <b>43</b> |
| <b>CPP-005 set/multiset/map/unordered_map</b>           | <b>45</b> |
| <b>CPP-006 bitset 与位运算</b>                              | <b>47</b> |
| <b>CPP-007 坐标压缩 Compressor</b>                          | <b>49</b> |
| <b>CPP-008 long long 与 __int128 溢出处理</b>                | <b>51</b> |
| <b>CPP-009 常见 RE/WA 语言坑清单</b>                           | <b>53</b> |
| <b>v0.2 本卷例题训练区</b>                                     | <b>54</b> |
| V01-EX01 班级成绩汇总 . . . . .                               | 55        |
| V01-EX02 商品小票打印 . . . . .                               | 56        |
| V01-EX03 EOF 单词频率表 . . . . .                            | 57        |
| V01-EX04 通讯录号码查询 . . . . .                              | 58        |
| V01-EX05 任务优先级调度 . . . . .                              | 59        |
| V01-EX06 整数去重排序 . . . . .                               | 61        |
| V01-EX07 排序后找第一个不小于目标的位置 . . . . .                      | 62        |
| V01-EX08 学生成绩排行 . . . . .                               | 63        |
| V01-EX09 整行记录统计 . . . . .                               | 64        |
| V01-EX10 闭区间计数查询 . . . . .                              | 65        |
| 第 9 卷: Python 互补卷例题 . . . . .                           | 66        |
| V01-CEX01 多行键值记录合并 . . . . .                            | 66        |
| V01-CEX02 固定宽度成绩表 . . . . .                             | 67        |
| V01-CEX03 哈希表加堆 TopK . . . . .                          | 68        |
| V01-CEX04 pair 排序合并区间 . . . . .                         | 69        |
| V01-CEX05 排序后二分查询 . . . . .                             | 69        |

## 第 1 卷 C++17 / STL / 输入输出

### 考试速查定位

#### 第 1 卷 C++17 / STL / 输入输出

- 这是第几卷: 01。
- 主要查什么: 主查 C++ 现场写法: 标准头文件、输入输出、保留小数、右对齐、EOF、string 成员函数、STL 容器和常用算法。
- 什么时候翻: 卡在语法、容器、string、格式化输出、EOF、快读快写时翻。
- 题面关键词: cin/cout; scanf/printf; getline; EOF; string; vector; priority\_queue; unordered\_map; sort; lower\_bound。

这一卷解决语言和 STL 层面的稳定性: 输入输出、字符串、容器、算法函数、排序二分、队列堆、映射、位运算、压缩、整数和常见 RE/WA。

核心口令:

允许 using namespace std。

数组默认 1-index。

字符串保持 0-index。

答案和距离用 long long。

容器访问前查 empty。

排序二分必须同一顺序。

### 头文件安全包

默认用 GNU g++ 时可以写 #include <bits/stdc++.h>。如果现场环境不支持, 直接翻 CPP-001, 把第一行替换成“标准头文件安全包”; 不要现场凭记忆一个个补头文件。

### 输入输出索引表

| 需求       | 优先查              | 常用写法                                             | 坑点                           |
|----------|------------------|--------------------------------------------------|------------------------------|
| 普通读写     | CPP-001 / CPP-10 | ios::sync_with_stdio(false)<br>cin.tie(nullptr); | 关闭同步后不要混用<br>scanf/printf    |
| 传统格式化    | CPP-10           | scanf/printf, %.2f, %04d                         | double 用 %lf 读、%f 输出         |
| 读整行/含空格  | CPP-10 / CPP-011 | getline(cin, line)                               | 前面用过 cin >> x 要 ignore<br>换行 |
| 快读快写     | CPP-10           | readInt(x), writeInt(x)                          | 只在数据极大时使用, 注意 EOF            |
| 小数/对齐/补零 | CPP-10           | fixed << setprecision(2),<br>setw                | setw 只影响下一个输出项               |

### STL 函数索引表

| 需求      | 优先查                                      | 常用写法                                        | 坑点                                 |
|---------|------------------------------------------|---------------------------------------------|------------------------------------|
| 字符串成员函数 | CPP-011 = string API;<br>STR-01 = 字符串题路由 | substr/find/rfind/insert/erase/replace      | erase 是 O(n)                       |
| 顺序容器    | CPP-013                                  | vector/deque/list/array                     | reserve 不改变 size, list 不能下标        |
| 优先队列    | CPP-013 / CPP-004                        | priority_queue<T,<br>vector<T>, greater<T>> | 默认最大堆, 小根堆写法较长                     |
| 哈希表     | CPP-013 / CPP-005                        | unordered_map、<br>unordered_set             | 大量插入前 max_load_factor<br>+ reserve |
| 排序与比较   | CPP-012 / CPP-003                        | sort(v.begin(), v.end(),<br>cmp)            | cmp 不能写 <=                         |
| 去重      | CPP-012                                  | sort +<br>erase(unique(...))                | unique 不会真的删尾部                     |
| 删除指定值   | CPP-012 / CPP-013                        | erase(remove(...), end)                     | 只适合顺序容器                            |
| 二分查找    | CPP-012 / CPP-003                        | lower_bound/upper_bound/binary_search       | 目标区间必须已排序                          |
| 全排列     | CPP-012 / BRUTE-03                       | next_permutation                            | 初始序列先排序                            |
| 数值工具    | CPP-012 / MATH-01                        | iota/accumulate/partial_sum                 | accumulate 初值写 0LL                 |
| 位集合     | CPP-006                                  | bitset<N>                                   | N 必须编译期常量                          |

### 模块目录

| 模块                                  | 内容                                                            |
|-------------------------------------|---------------------------------------------------------------|
| CPP-001-main-io.md                  | C++17 主骨架与输入输出                                                |
| CPP-10-io-formatting.md             | C++10: 输入输出与格式化                                               |
| CPP-002-basic-containers.md         | C++002 vector/string/pair/tuple 基础容器                          |
| CPP-011-string-reference.md         | C++011 string 考场速查                                            |
| STR-01-basic-operations.md          | STR-01 string 常用操作                                            |
| CPP-013-stl-containers-reference.md | C++013 STL 容器成员函数速查                                           |
| CPP-012-stl-algorithms-reference.md | C++012 STL 常用算法速查                                             |
| CPP-003-sort-bounds.md              | C++003 sort/lambda/比较函数/number_lower_bound/number_upper_bound |
| CPP-004-queues-stacks-heaps.md      | C++004 queue/deque/stack/priority_queue                       |
| CPP-005-associative-containers.md   | C++005 set/multiset/map/unordered_map                         |
| CPP-006-bitset-bit-operations.md    | C++006 bitset 与位运算                                            |
| CPP-007-coordinate-compression.md   | C++007 坐标压缩 Compressor                                        |
| CPP-008-integers-overflow.md        | C++008 long long 与 __int128 溢出处理                              |
| CPP-009-common-re-wa-pitfalls.md    | C++009 常见 RE/WA 语言坑清单                                         |

## CPP-001 C++17 主骨架与输入输出

模块编号: CPP-001

模块名称: C++17 主骨架与输入输出

标签: C++17、标准输入输出、多组数据、EOF、getline、统一类型

一句话用途: 每道题先抄这一页, 得到稳定的 main + solve 外壳和常用读入方式。

题面触发词: 多组数据、直到输入结束、每行一个字符串、字符串可能含空格、输出每组答案、标准输入输出。

什么时候用: 所有 C++17 题目开写前; 需要统一 long long、换行、数组 1-index 和多组数据结构时。

不要什么时候用: 交互题需要按题面及时刷新输出; 题面明确给出完全不同的框架时。

复杂度: 读入输出本身是  $O(\text{输入规模} + \text{输出规模})$ 。

数据范围参考: 数值、权值、答案、计数可能到  $1e18$  时用 long long; 下标、点数、边数通常用 int。

依赖的标准容器: vector、string、pair; 全卷统一 ll = long long。

输入如何整理: 数组默认整理成 vector<ll> a(n + 1), 使用下标 1..n; 字符串保留 C++ 默认 0..len-1。

接口:

- void solve(): 处理一组数据。
- int main(): 只负责加速 IO、选择单组/多组/EOF 模式、调用 solve()。
- read\_case(): 遇到 EOF 多组时可写成返回 bool 的读入函数。

输出能力: 使用 cout << ans << '\n'; 大量输出时也用 '\n', 避免频繁强制刷新。

下游可接: 所有算法模块、数据结构模块、图论模块、DP 模块。

可拼接模块: CPP-002 基础容器、CPP-008 整数溢出、CPP-009 语言坑。

模板代码:

考场默认可以用 GNU g++ 时, 第一行写:

```
#include <bits/stdc++.h>
```

如果现场环境不支持 bits/stdc++.h, 把它替换成下面这个“标准头文件安全包”。这组头文件覆盖本资料中常用的 STL 容器、算法、数学、格式化、字符串流、快读快写和断言。

```
#include <algorithm>
#include <array>
#include <bitset>
#include <cassert>
#include <cctype>
#include <climits>
#include <cmath>
```

```

#include <complex>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <deque>
#include <functional>
#include <iomanip>
#include <iostream>
#include <iterator>
#include <limits>
#include <list>
#include <map>
#include <numeric>
#include <queue>
#include <set>
#include <sstream>
#include <stack>
#include <string>
#include <tuple>
#include <type_traits>
#include <unordered_map>
#include <unordered_set>
#include <utility>
#include <vector>

```

头文件速查:

| 用到的东西                               | 标准头                                         |
|-------------------------------------|---------------------------------------------|
| cin/cout/ios                        | <iostream>                                  |
| printf/scanf/getchar/putchar        | <cstdio>                                    |
| vector/string/array/list/deque      | <vector>、<string>、<array>、<list>、<deque>    |
| queue/stack/priority_queue          | <queue>、<stack>                             |
| set/map/unordered_map/unordered_set | <set>、<map>、<unordered_map>、<unordered_set> |
| sort/lower_bound/unique/min/max     | <algorithm>                                 |
| iota/accumulate/partial_sum/gcd/lcm | <numeric>                                   |
| pair/tuple                          | <utility>、<tuple>                           |
| greater/function/hash               | <functional>                                |
| bitset                              | <bitset>                                    |
| numeric_limits/LLONG_MAX/INT_MAX    | <limits>、<climits>                          |
| setw/setfill/setprecision/fixed     | <iomanip>                                   |
| stringstream                        | <sstream>                                   |
| isdigit/tolower                     | <cctype>                                    |
| sqrt/fabs                           | <cmath>                                     |
| assert                              | <cassert>                                   |
| make_unsigned 快写模板                  | <type_traits>                               |

完整主骨架:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

const int INF = 1000000000;
const ll LINF = 4'000'000'000'000'000'000LL;

void solve() {
    int n;
    cin >> n;
}

```

```

vector<ll> a(n + 1);
for (int i = 1; i <= n; i++) cin >> a[i];

ll sum = 0;
for (int i = 1; i <= n; i++) sum += a[i];

cout << sum << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();          // 单组数据

    return 0;
}

```

调用示例:

// 题面写“第一行  $T$ , 表示  $T$  组数据”时, 把 `main` 中的 `solve()` 换成:

```

int T;
cin >> T;
while (T--) {
    solve();
}

```

// 题面写“直到输入结束”时, 常用写法:

```

int n, m;
while (cin >> n >> m) {
    // 处理这一组 n, m
}

```

// 题面有整行字符串且前面刚用过 `cin >> x`:

```

cin.ignore(numeric_limits<streamsize>::max(), '\n');
string line;
getline(cin, line);

```

常见坑:

- 如果 `#include <bits/stdc++.h>` 不可用, 不要现场猜头文件; 直接替换成上面的标准头文件安全包。
- 题面没有  $T$  时不要强读  $T$ 。
- `cin >> s` 读不到空格; 含空格必须用 `getline(cin, s)`。
- `cin >> x` 后立刻 `getline`, 要先用 `ignore` 吃掉行尾换行。
- `endl` 会刷新缓冲, 大量输出时可能慢; 普通换行用 `'\n'`。
- 不要依赖本地文件输入, OJ 默认使用标准输入输出。
- 不要写编译器私有优化指令; 纸质模板以稳为主。
- 不要用宏把所有 `int` 替换成 `long long`, 下标和容器大小仍用 `int` 更稳。

暴力/部分分替代: 先把读入和输出跑通; 不会算法时也先输出合法格式, 避免格式错误。

升级方向: 把 `solve()` 内部替换为暴力、记忆化、DP、图论或数据结构正解; `main` 通常不动。

最小测试样例:

输入  
3  
10 20 30

输出  
60

---

## CPP-10: 输入输出与格式化

模块编号: CPP-10

模块名称：输入输出与格式化

标签：[C++17][输入输出][格式化][快速快写][小数][对齐]

一句话用途：把 cin/cout、scanf/printf、快速快写和常见输出格式统一成考场可抄片段，避免格式错误丢分。

题面触发词：

- 多组数据、读到 EOF。
- 保留小数、四舍五入、允许误差。
- 右对齐、左对齐、补零、表格输出。
- 输入含空格字符串、整行、字符网格。
- 输入里有括号、逗号、冒号等固定标点。

什么时候用：

- 每道题开写前确认输入输出形态。
- 样例输出有固定小数位、列宽、补零或特殊标点。
- 数据量很大，普通 cin/cout 可能偏慢。
- 需要读整行或混合读数字和字符串。

不要什么时候用：

- 已经使用关闭同步的 cin/cout 时，不要再混用 scanf/printf。
- 普通数据量题不要先写复杂快读，cin/cout 关闭同步更稳。
- 字符串中包含空格时，不要用 cin >> s 期待读整行。

复杂度：

- 输入输出本身按字符数线性。
- 快速快写常数更小，但代码更长。

依赖的标准容器：

- string、stringstream、vector。
- 格式化输出使用 <iomanip>，bits/stdc++.h 已包含。

接口：

```
cin/cout: ios::sync_with_stdio(false); cin.tie(nullptr)
scanf/printf: %d, %lld, %lf, %.2f, %04d
readInt(x): 快读整数, 读到 EOF 返回 false
writeInt(x, end): 快写整数
getline(cin, line): 读整行
```

常见坑：

- endl 会刷新，普通换行用 '\n'。
- fixed << setprecision(2) 是小数点后 2 位，单独 setprecision(2) 是总有效数字。
- setw 只影响下一个输出项，setfill/left/right 会持续生效。
- scanf 读 double 用 %lf，printf 输出 double 用 %f。
- getline 前如果刚用过 cin >> x，要先 ignore 掉行尾换行。

暴力/部分替代：

- 复杂格式看不懂时，先整行 getline，再用 stringstream 或手动扫描。
- 输出格式不确定时，优先照样例保持空格和换行，避免额外调试输出。

## 考场目标

输入输出的目标不是优雅，是稳定拿分：

选一套 IO。

多组和 EOF 判断写对。

小数位数按题面。

空格、换行、补零、对齐按样例。

不要 freopen，不要文件 IO，不要 #pragma。

本模块默认 C++17、ACM/机考标准输入输出，允许 using namespace std。

## 1. 先选 IO 套路

| 场景            | 推荐                           |
|---------------|------------------------------|
| 普通题、字符串多      | cin/cout + 关闭同步              |
| 传统格式化多、全是基础类型 | scanf/printf                 |
| 极大整数输入输出      | 自写快读快写                       |
| 要读整行、含空格字符串   | getline                      |
| 复杂标点格式        | char 吃掉标点, 或整行后 stringstream |

口令:

不要把关闭同步后的 cin/cout 和 scanf/printf 混用。  
endl 会刷新, 普通换行用 '\n'。  
setw 只影响下一个输出项, setfill/left/right 会持续生效。

## 1A. 分隔方式总表

最重要的判断: 题目输入到底是 “token 流”, 还是 “每一行有特殊含义”。

| 输入形态                 | C++ 推荐                                       | 说明                     |
|----------------------|----------------------------------------------|------------------------|
| 整数/单词由空格、换行、Tab 任意分隔 | cin >> x                                     | >> 会自动跳过所有空白, 空格和换行等价  |
| 固定个数数组, 但可能跨多行       | for (...) cin >> a[i]                        | 不要按行读, 直接按 token 读最稳   |
| 第一行一个句子, 含空格         | getline(cin, line)                           | cin >> s 只能读到第一个空格     |
| 前面读过数字, 后面马上读整行      | cin.ignore(..., '\n');<br>getline(cin, line) | 吃掉上一行末尾换行              |
| 允许跳过空行和前导空白后读一行      | getline(cin >> ws, line)                     | 会丢掉行首空格; 题目要保留行首空格时不要用 |
| 一行里有不定个整数            | getline + stringstream                       | 行边界有意义时用               |
| 逗号/冒号/括号固定格式         | char 接标点, 或整行扫描                              | 如 (12,34)、key: value   |
| CSV/JSON/脚本等半结构化文本   | 整段 getline/读全文                               | 翻 SIM-03/04/05         |

口令:

只要题面说 “若干整数/字符串”, 且没说每行含义不同, 就用 cin >>。  
只有行本身有意义, 或字符串可能含空格, 才用 getline。

### 空格和换行等价的例子

下面两份输入对 cin >> n >> a[1] >> a[2] >> a[3] 完全一样:

```
3
10 20 30

3 10
20
30
```

代码:

```
int n;
cin >> n;
static int a[100005];
for (int i = 1; i <= n; i++) cin >> a[i];
```

### 行边界有意义的例子

每一行一个表达式、日志、名字、句子时, 不能只用 cin >>。

```
int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');

for (int i = 1; i <= n; i++) {
    string line;
    getline(cin, line); // line 可以包含空格, 也可以是空行
    // process line
}
```



## 2. cin/cout 常用片段

快速 cin/cout 骨架

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        ll x;
        cin >> x;
        sum += x;
    }

    cout << sum << '\n';
    return 0;
}
```

### 多组数据

题面给 T:

```
int T;
cin >> T;
while (T--) {
    solve();
}
```

题面没给 T, 读到 EOF:

```
int n, m;
while (cin >> n >> m) {
    // write one case here
}
```

### EOF 读到输入末尾就停止

EOF 题的口令:

不要先判断 `cin.eof()`。  
直接尝试读取; 读成功就处理, 读失败就停止。

读未知个整数, 直到输入结束:

```
long long x;
while (cin >> x) {
    // use x
}
```

每组两个数, 直到输入结束:

```
int n, m;
while (cin >> n >> m) {
    cout << (n + m) << '\n';
}
```

每组结构复杂时, 写 `read_case()`:

```
#include <bits/stdc++.h>
using namespace std;
```

```

const int MAXN = 100000 + 5;
const int MAXM = 200000 + 5;

int n, m;
int a[MAXN], u[MAXM], v[MAXM];

bool read_case() {
    if (!(cin >> n >> m)) return false;
    for (int i = 1; i <= n; i++) cin >> a[i];
    for (int i = 1; i <= m; i++) cin >> u[i] >> v[i];
    return true;
}

void solve_one_case() {
    long long sum = 0;
    for (int i = 1; i <= n; i++) sum += a[i];
    cout << sum << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    while (read_case()) {
        solve_one_case();
    }
    return 0;
}

```

注意：read\_case() 里用到的 n/m/a/u/v 可以是全局变量，也可以改成传引用。关键是第一句先判断这一组是否真的存在。

如果题面给的是“第一行 T”，不要写 EOF 循环；如果题面没给 T，不要强读 T。

### 小数保留位数

```

double x;
cin >> x;

cout << fixed << setprecision(2) << x << '\n'; // 保留 2 位小数

```

提醒：

fixed + setprecision(2): 小数点后 2 位。

只有 setprecision(2): 总有效数字 2 位。

### 右对齐、左对齐、补零

```

int x = 7;
string name = "Tom";

cout << setw(5) << x << '\n'; // 默认右对齐: 7
cout << right << setw(5) << x << '\n'; // 右对齐
cout << left << setw(10) << name << "!" << '\n'; // 左对齐

cout << right << setfill('0') << setw(4) << x << '\n'; // 0007
cout << setfill(' '); // 记得恢复空格填充

```

表格输出：

```

cout << left << setw(12) << "name"
    << right << setw(5) << "score" << '\n';

cout << left << setw(12) << name
    << right << setw(5) << 98 << '\n';

```

### 3. scanf/printf 常用片段

#### 基础骨架

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        ll x;
        if (scanf("%lld", &x) != 1) return 0;
        sum += x;
    }

    printf("%lld\n", sum);
    return 0;
}
```

#### 常用格式符

| 类型           | scanf | printf |
|--------------|-------|--------|
| int          | %d    | %d     |
| long long    | %lld  | %lld   |
| double       | %lf   | %f     |
| char         | %c    | %c     |
| C 字符串 char[] | %100s | %s     |

scanf 读 char[] 时要给宽度, 例如 char s[105]; scanf("%100s", s);。裸 %s 在输入过长时可能越界; 考场更推荐 string s; cin >> s;。

#### 小数、宽度、补零

```
double x = 3.14159;
int a = 7;

printf("%.2f\n", x); // 3.14
printf("%5d\n", a); // 宽度 5, 右对齐:    7
printf("%04d\n", a); // 宽度 4, 前面补 0: 0007
```

多组数据:

```
int T;
scanf("%d", &T);
while (T--) {
    solve();
}
```

直到 EOF:

```
int n, m;
while (scanf("%d%d", &n, &m) == 2) {
    // write one case here
}
```

### 4. 自写快读快写

只在输入输出量特别大、且主要是整数时使用。用了它就全程用它, 不要再混 cin。

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

template <class T>
bool readInt(T &x) {
    int c = getchar();
    if (c == EOF) return false;

    while (c != '-' && (c < '0' || c > '9')) {
        c = getchar();
        if (c == EOF) return false;
    }

    int neg = 0;
    if (c == '-') {
        neg = 1;
        c = getchar();
    }

    using U = make_unsigned_t<T>;
    U y = 0;
    while (c >= '0' && c <= '9') {
        y = y * 10 + (U)(c - '0');
        c = getchar();
    }

    if (neg) x = (T)(U(0) - y);
    else x = (T)y;
    return true;
}

template <class T>
void writeInt(T x, char end = '\n') {
    if (x == 0) {
        putchar('0');
        putchar(end);
        return;
    }

    using U = make_unsigned_t<T>;
    U y;
    if (x < 0) {
        putchar('-');
        y = U(0) - (U)x; // 避免 LLONG_MIN 取负溢出
    } else {
        y = (U)x;
    }

    char s[50]; // Long Long 足够; __int128 请用 CPP-008 的专门打印模板
    int top = 0;
    while (y > 0) {
        s[top++] = char('0' + y % 10);
        y /= 10;
    }
    while (top--) putchar(s[top]);
    putchar(end);
}

int main() {
    int n;
    if (!readInt(n)) return 0;

```

```

    ll sum = 0;
    for (int i = 1; i <= n; i++) {
        ll x = 0;
        if (!readInt(x)) return 0;
        sum += x;
    }

    writeInt(sum);
    return 0;
}

```

EOF 多组:

```

int n, m;
while (readInt(n) && readInt(m)) {
    // write one case here
}

```

scanf 版 EOF:

```

int n, m;
while (scanf("%d%d", &n, &m) == 2) {
    printf("%d\n", n + m);
}

```

scanf 返回成功读到的变量个数; 读两个整数就检查是否等于 2。不要只写 != EOF, 因为输入残缺时可能只读到 1 个。

## 5. 字符、整行与换行处理

### 读一个非空白字符

cin >> c 会跳过空格和换行:

```

char c;
cin >> c;

```

scanf(" %c", &c) 前面的空格会跳过所有空白:

```

char c;
scanf(" %c", &c);

```

### 读下一个原始字符

需要读空格或换行本身:

```

char c;
cin.get(c);

char c;
scanf("%c", &c);

```

### 读整行

```

string line;
getline(cin, line);

```

如果前面刚用过 cin >> n, 要先吃掉本行剩余换行:

```

int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');

```

```

string line;
getline(cin, line);

```

读到 EOF 的整行:

```

string line;
while (getline(cin, line)) {

```

```

    // process line
}

```

空行不是 EOF。getline 读到空行时 line == "" 但循环仍然成立；只有真的没有下一行时循环才结束。

读完整剩余输入，包括换行和空格：

```
string text((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());
```

如果前面刚读过一个模式名，还要读后面整段文本：

```

string mode;
cin >> mode;
cin.ignore(numeric_limits<streamsize>::max(), '\n');

string text((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());

```

## 6. 复杂格式处理

**标点固定：用 char 接住**

输入形如 (12,34)：

```

char l, comma, r;
int x, y;
cin >> l >> x >> comma >> y >> r;

```

或者：

```

int x, y;
scanf(" (%d,%d)", &x, &y);

```

**一行里有不定个整数**

```

string line;
while (getline(cin, line)) {
    stringstream ss(line);
    int x;
    while (ss >> x) {
        // use x
    }
}

```

**简单逗号分隔**

只适用于题目保证没有引号包裹、字段中不含逗号的简单格式。真正 CSV 翻 SIM-04。

```

string line;
getline(cin, line);

stringstream ss(line);
string cell;
vector<string> cells(1); // 1-index
while (getline(ss, cell, ',')) {
    cells.push_back(cell);
}

```

**key=value 或 key: value**

```

string line;
getline(cin, line);

int p = line.find('=');
if (p != (int)string::npos) {
    string key = line.substr(0, p);
    string value = line.substr(p + 1);
}

```

冒号同理把 '=' 改成 ':'。若要去掉两侧空格，翻 STR-01 或手写 trim。

### 读带空格的名字和值

输入每行形如 Tom Hanks 98, 最后一个分数:

```
string line;
getline(cin, line);

stringstream ss(line);
vector<string> parts;
string word;
while (ss >> word) parts.push_back(word);
if (!parts.empty()) {
    int score = stoi(parts.back());
    parts.pop_back();

    string name;
    for (int i = 0; i < (int)parts.size(); i++) {
        if (i) name += ' ';
        name += parts[i];
    }
}
```

### 网格读入

无空格字符网格:

```
int n, m;
cin >> n >> m;
vector<string> g(n + 1);
for (int i = 1; i <= n; i++) {
    string row;
    cin >> row;
    g[i] = " " + row; // 之后访问 g[i][j], i=1..n, j=1..m
}
```

有空格分隔的字符网格:

```
const int MAXN = 1000 + 5;
const int MAXM = 1000 + 5;
static char g[MAXN][MAXM];

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        cin >> g[i][j];
    }
}
```

## 7. 提交前格式检查

题面有没有 T? 没有就别强读 T。

EOF 题 while 条件是否判断了读入成功?

小数是保留几位, 还是允许误差?

每个答案后有没有换行?

多个数之间是空格还是换行?

printf 的 long long 是否用 %lld?

scanf 读 double 是否用 %lf?

getline 前是否处理了上一次 cin 留下的换行?

setfill('0') 后是否恢复 setfill(' ')?

有没有 endl、调试输出、freopen、文件 IO、#pragma?

---

## CPP-002 vector/string/pair/tuple 基础容器

模块编号: CPP-002

模块名称: vector / string / pair / tuple 基础容器

标签: vector、string、pair、tuple、结构化绑定、1-index 数组

一句话用途: 把题目输入整理成全卷统一的数组、字符串、二元组和多元组形态。

题面触发词: 数组、序列、字符串、坐标、物品属性、二元关系、三元状态、按多个字段保存。

什么时候用: 需要保存一串数、一张表、一个字符串、若干 (值, 编号) 或 (x, y, w) 记录时。

不要什么时候用: 需要频繁在中间插入删除且数据量很大时, 优先考虑链式结构或其他专用数据结构; 需要自动去重/排序时用 set/map。

复杂度: vector 尾部插入均摊  $O(1)$ , 随机访问  $O(1)$ ; string 下标访问  $O(1)$ ; pair/tuple 只是打包字段。

数据范围参考:  $n \leq 2e5$  常用 vector; 二维  $n*m$  容器先估算内存, int 约 4 字节, long long 约 8 字节。

依赖的标准容器: vector、string、pair、tuple。

输入如何整理:

- 数组: 上限明确时优先 `static ll a[MAXN]`; 运行时尺寸才用 `vector<ll> a(n + 1)`, 都使用 `1..n`。
- 字符串: `string s`, 使用 C++ 自然 `0..s.size()-1`。
- 二元记录: `vector<pair<ll, int>> v` 保存 (值, 原编号)。
- 多字段记录: 简单时用 tuple, 字段含义复杂时改 struct。

接口:

- `a[i]`: 访问 1-index 数组元素。
- `s[i]`: 访问 0-index 字符。
- `v.push_back(x)`: 尾部加入; 若元素有题面编号, 记录里的 id 仍保存 `1..n`。
- `v.size()`: 元素个数, 比较或循环时常转成 int。
- `auto [x, y] = p`: 拆 pair。
- `auto [x, y, z] = t`: 拆 tuple。

输出能力: 按题面输出数组、字符串、记录的字段; 容器本身不能直接 cout, 要循环输出。

下游可接: 排序、二分、前缀和、树状数组、线段树、DP、图论边表。

可拼接模块: CPP-003 排序二分、CPP-007 坐标压缩、CPP-008 整数溢出。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    string s;
    cin >> n >> s;

    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    vector<pair<ll, int>> value_id;
    for (int i = 1; i <= n; i++) {
        value_id.push_back({a[i], i});
    }

    if (n == 0 || s.empty() || value_id.empty()) return 0;

    tuple<int, int, ll> state = {1, (int)s.size(), a[1]};
    auto [l, r, val] = state;

    cout << s[0] << ' ' << value_id[0].first << ' ' << value_id[0].second << '\n';
```



```

    cout << l << ' ' << r << ' ' << val << '\n';

    return 0;
}

```

调用示例:

```

const int MAXN = 1000 + 5;
const int MAXM = 1000 + 5;
static int grid[MAXN][MAXM];

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        cin >> grid[i][j];
    }
}

vector<tuple<int, int, ll>> edges;
edges.push_back({u, v, w});
for (auto [from, to, weight] : edges) {
    // 使用 from, to, weight
}

```

常见坑:

- 普通数组/DP 表不要写 `vector<int> a(n)` 后再按 `1..n` 用; 上限明确直接全局静态数组, 动态 `vector` 也开 `n + 1`。
- `s.size()` 是无符号类型, 倒着循环时先写 `int len = (int)s.size()`。
- 空 `vector/string` 不能访问 `[0]`、`back()`。
- `pair` 默认先按 `first` 排, 再按 `second` 排。
- `tuple` 字段多了可读性会变差, 复杂记录建议写 `struct`。
- `vector` 扩容可能让旧引用、旧指针失效; 保存下标通常更稳。

暴力/部分替代: 小数据直接用 `vector` 暴力双循环; 记录原编号时用 `pair` 避免排序后丢失位置。

升级方向: `vector` 接前缀和/树状数组/线段树; `pair/tuple` 接排序、离线查询、Kruskal 边表。

最小测试样例:

输入  
3 abc  
5 6 7

输出  
a 5 1  
1 3 5

## CPP-011 string 考场速查

模块编号: CPP-011

模块名称: string 考场速查与常用代码

标签: [C++17][string][字符处理][子串查找][getline][类型转换]

一句话用途: 把 C++ string 的构造、访问、修改、查找、比较、转换和字符统计整理成考场可直接抄的可靠片段。

题面触发词:

- 字符串、单词、整行文本、含空格输入。
- 子串、前缀、后缀、查找最后一次出现。
- 字典序最小/最大、按字符串排序。
- 删除、插入、替换、拼接。
- 数字字符串转整数、整数转字符串。
- 判断字符是否为数字/字母、大小写转换、统计字符频次。

什么时候用:

- 只需要 STL 基础字符串操作, 不需要 KMP、Hash、Trie。

- 字符串长度和操作次数不大, 允许 insert/erase/replace/substr 复制或移动字符。
- 题目要求模拟编辑字符串、统计字符、分割单词、做简单查找。

不要什么时候用:

- 长串反复模式匹配, 优先 STR-02 KMP/Z。
- 大量任意子串相等判断, 优先 STR-03 Rolling Hash。
- 大量在字符串中间插入删除, string 会整体移动字符, 可能 TLE。

复杂度:

- size/empty/operator[]:  $O(1)$ 。
- push\_back/pop\_back/back/front: 尾部操作通常  $O(1)$ 。
- insert/erase/replace: 最坏  $O(n)$ , 因为要移动后面的字符。
- substr(pos, len):  $O(len)$ , 会复制新字符串。
- find/rfind/compare: 和参与比较/查找的字符数相关, 频繁大规模查找不要当作线性算法保证。

索引约定:

C++ string 自然 0-index: 合法位置是  $0..s.size()-1$ 。

题面第  $k$  个字符若是 1-index, 写  $k--$  后再访问。

substr(pos, len): pos 是起点, len 是长度, 不是右端点。

题面 1-index 闭区间  $[l, r]$ :  $s.substr(l - 1, r - l + 1)$ 。

依赖的标准容器:

- string。
- vector<string>。
- 字符计数常用 vector<int>(256) 或 array<int, 26>。
- 分割整行可用 stringstream。

接口速查:

| 需求          | 写法                        |
|-------------|---------------------------|
| 构造空串        | string s;                 |
| 构造重复字符      | string s(n, 'a');         |
| 从 C 风格字符串构造 | string s = "abc";         |
| 拼接          | s += t; 或 s = a + b;      |
| 长度          | int n = (int)s.size();    |
| 判空          | s.empty()                 |
| 下标访问        | s[i]                      |
| 首尾字符        | s.front()、s.back(), 先保证非空 |
| 尾部加入        | s.push_back(c); 或 s += c; |
| 尾部删除        | s.pop_back();, 先保证非空      |
| 插入          | s.insert(pos, t);         |
| 删除          | s.erase(pos, len);        |
| 替换          | s.replace(pos, len, t);   |
| 截取          | s.substr(pos, len);       |
| 从左查找        | s.find(t)                 |
| 从右查找        | s.rfind(t)                |
| 比较          | s.compare(t) 或 $s < t$    |
| 转整数         | stoi(s)、stoll(s)          |
| 转字符串        | to_string(x)              |

## 1. 构造、长度、访问

```
string a;           // 空串
string b = "abc";   // "abc"
string c(5, 'x');   // "xxxxx"
string d = b + c;   // 拼接
```

```
int n = (int)b.size(); // 推荐转 int, 避免无符号坑
if (!b.empty()) {
    char first = b.front();
    char last = b.back();
    if ((int)b.size() > 1) {
```

```

        char mid = b[1]; // 0-index, b[1] 是第二个字符
    }
}

```

遍历:

```

string s = "abc";

for (int i = 0; i < (int)s.size(); i++) {
    cout << i << ' ' << s[i] << '\n';
}

for (char c : s) {
    cout << c << '\n';
}

```

## 2. 修改: push\_back / pop\_back / insert / erase / replace

```

string s = "abc";

s.push_back('d');           // "abcd"
s += 'e';                   // "abcde"
s += "fg";                   // "abcdefg"

if (!s.empty()) {
    s.pop_back();           // "abcdef"
}

s.insert(3, "XXX");          // 在下标 3 前插入: "abcXXXdef"
s.erase(3, 3);               // 从下标 3 开始删 3 个: "abcdef"
s.replace(1, 3, "00");        // 从下标 1 开始替换 3 个: "a00ef"

```

安全删除某个字符:

```

string s = "abcde";
int pos = 2; // 删除 'c'
if (0 <= pos && pos < (int)s.size()) {
    s.erase(pos, 1); // "abde"
}

```

删除所有某字符, 推荐新建结果串, 比反复 erase 更稳:

```

string remove_char(const string &s, char bad) {
    string res;
    for (char c : s) {
        if (c != bad) res.push_back(c);
    }
    return res;
}

```

## 3. substr / find / rfind

```

string s = "abracadabra";

string t1 = s.substr(0, 4); // "abra"
string t2 = s.substr(3);    // 从下标 3 到结尾: "acadabra"

size_t p = s.find("ra");    // 第一次出现位置: 2
if (p != string::npos) {
    cout << "found at " << p << '\n';
}

size_t q = s.rfind("ra");   // 最后一次出现位置: 9
if (q != string::npos) {
    cout << "last at " << q << '\n';
}

```

从某个位置开始找：

```
string s = "aaaa";
size_t p = s.find("aa", 1); // 从下标 1 开始找, 结果 1
```

统计模式串出现次数，允许重叠：

```
int count_occurrence_overlap(const string &s, const string &pat) {
    if (pat.empty()) return 0;
    int ans = 0;
    size_t pos = s.find(pat);
    while (pos != string::npos) {
        ans++;
        pos = s.find(pat, pos + 1);
    }
    return ans;
}
```

不允许重叠：

```
int count_occurrence_no_overlap(const string &s, const string &pat) {
    if (pat.empty()) return 0;
    int ans = 0;
    size_t pos = s.find(pat);
    while (pos != string::npos) {
        ans++;
        pos = s.find(pat, pos + pat.size());
    }
    return ans;
}
```

#### 4. compare 与字典序

string 默认按字典序比较，和字典中排序类似：从左到右找第一个不同字符，字符小的字符串更小；若前面都相同，短的更小。

```
string a = "abc";
string b = "abd";
string c = "abcde";
```

```
cout << (a < b) << '\n'; // true, 因为 'c' < 'd'
cout << (a < c) << '\n'; // true, 因为 a 是 c 的前缀且更短
```

```
int x = a.compare(b); // x < 0 表示 a < b
int y = b.compare(a); // y > 0 表示 b > a
int z = a.compare("abc"); // z == 0 表示相等
```

排序字符串：

```
vector<string> v = {"ba", "ab", "aa"};
sort(v.begin(), v.end()); // "aa", "ab", "ba"
```

自定义：先按长度，再按字典序：

```
sort(v.begin(), v.end(), [](const string &x, const string &y) {
    if (x.size() != y.size()) return x.size() < y.size();
    return x < y;
});
```

#### 5. getline 混 cin

cin >> x 会留下行尾换行；接着 getline 会先读到这个空行。解决：在第一次 getline 前吃掉换行。

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');

for (int i = 0; i < n; i++) {
    string line;
    getline(cin, line);
    cout << line << '\n';
}

return 0;
}

```

只读无空格字符串:

```

string s;
cin >> s;

```

读整行, 允许空格:

```

string line;
getline(cin, line);

```

## 6. stoi / stoll / to\_string

```

string a = "123";
string b = "-900000000000000";

```

```

int x = stoi(a);
long long y = stoll(b);

```

```

string sx = to_string(x);
string sy = to_string(y);

```

从字符串中扫描整数, 考场更常用手扫, 避免异常处理:

```

vector<long long> read_ints_from_line(const string &line) {
    vector<long long> nums;
    int n = (int)line.size();
    for (int i = 0; i < n; ) {
        while (i < n && !isdigit((unsigned char)line[i]) && line[i] != '-') i++;
        if (i >= n) break;

        int sign = 1;
        if (line[i] == '-') {
            sign = -1;
            i++;
        }

        long long x = 0;
        bool has_digit = false;
        while (i < n && isdigit((unsigned char)line[i])) {
            has_digit = true;
            x = x * 10 + (line[i] - '0');
            i++;
        }
        if (has_digit) nums.push_back(sign * x);
    }
    return nums;
}

```

提醒:

stoi 超出 int 范围或字符串不是合法数字会抛异常; 竞赛中若不想写 try/catch, 先保证输入合法。  
大整数用 stoll; 再大就按字符串处理或高精度。

## 7. 字符函数: isdigit / isalpha / tolower / toupper

这些函数来自 `<cctype>`, `bits/stdc++.h` 已包含。为了避免 `char` 为负导致未定义行为, 传参时推荐转 `unsigned char`。

```
char c = 'A';

if (isdigit((unsigned char)c)) {
    cout << "digit\n";
}
if (isalpha((unsigned char)c)) {
    cout << "letter\n";
}

char low = (char)tolower((unsigned char)c); // 'a'
char up = (char)toupper((unsigned char)low); // 'A'
```

整串转小写:

```
string to_lower_string(string s) {
    for (char &c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}
```

只保留字母数字并转小写:

```
string normalize_alnum_lower(const string &s) {
    string res;
    for (char c : s) {
        unsigned char uc = (unsigned char)c;
        if (isalnum(uc)) {
            res.push_back((char)tolower(uc));
        }
    }
    return res;
}
```

## 8. 按字符计数

小写字母计数:

```
array<int, 26> count_lower(const string &s) {
    array<int, 26> cnt{};
    for (char c : s) {
        if ('a' <= c && c <= 'z') {
            cnt[c - 'a']++;
        }
    }
    return cnt;
}
```

ASCII 字符计数:

```
vector<int> count_ascii(const string &s) {
    vector<int> cnt(256, 0);
    for (unsigned char c : s) {
        cnt[c]++;
    }
    return cnt;
}
```

判断两个小写字符串是否为异位词:

```
bool same_lower_multiset(const string &a, const string &b) {
    return count_lower(a) == count_lower(b);
}
```

## 9. split 思路

按空白切单词，最省事用 stringstream:

```
vector<string> split_by_space(const string &line) {
    stringstream ss(line);
    vector<string> words;
    string word;
    while (ss >> word) {
        words.push_back(word);
    }
    return words;
}
```

按指定分隔符切，例如逗号:

```
vector<string> split_by_char(const string &s, char sep) {
    vector<string> parts;
    string cur;
    for (char c : s) {
        if (c == sep) {
            parts.push_back(cur);
            cur.clear();
        } else {
            cur.push_back(c);
        }
    }
    parts.push_back(cur);
    return parts;
}
```

调用示例:

```
string line = "alice,bob,,tom";
vector<string> parts = split_by_char(line, ',');
// parts = {"alice", "bob", "", "tom"}
```

## 10. 可抄完整模板

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

string substr_lidx(const string &s, int l, int r) {
    return s.substr(l - 1, r - l + 1);
}

array<int, 26> count_lower(const string &s) {
    array<int, 26> cnt{};
    for (char c : s) {
        if ('a' <= c && c <= 'z') cnt[c - 'a']++;
    }
    return cnt;
}

vector<int> count_ascii(const string &s) {
    vector<int> cnt(256, 0);
    for (unsigned char c : s) cnt[c]++;
    return cnt;
}

string to_lower_string(string s) {
    for (char &c : s) c = (char)tolower((unsigned char)c);
    return s;
}
```

```

vector<string> split_by_space(const string &line) {
    stringstream ss(line);
    vector<string> words;
    string word;
    while (ss >> word) words.push_back(word);
    return words;
}

vector<string> split_by_char(const string &s, char sep) {
    vector<string> parts;
    string cur;
    for (char c : s) {
        if (c == sep) {
            parts.push_back(cur);
            cur.clear();
        } else {
            cur.push_back(c);
        }
    }
    parts.push_back(cur);
    return parts;
}

int count_occurrence_overlap(const string &s, const string &pat) {
    if (pat.empty()) return 0;
    int ans = 0;
    size_t pos = s.find(pat);
    while (pos != string::npos) {
        ans++;
        pos = s.find(pat, pos + 1);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s = "abracadabra";

    s.push_back('!');
    s.pop_back();
    s.insert(3, "X");
    s.erase(3, 1);
    s.replace(0, 4, "ABRA");

    cout << s.substr(0, 4) << '\n';
    cout << count_occurrence_overlap(s, "ra") << '\n';

    vector<string> words = split_by_space("one two three");
    for (string w : words) cout << w << '\n';

    int x = stoi("123");
    long long y = stoll("1234567890123");
    cout << to_string(x + 1) << ' ' << to_string(y) << '\n';

    return 0;
}

```

## 11. 常见坑

- string 是 0-index; 题面位置常是 1-index, 访问前先减一。



- `s[i]` 不检查越界; `s.at(i)` 会检查但较少用。考场访问前确认  $0 \leq i < (\text{int})s.size()$ 。
- 空串不能用 `front/back/pop_back`。
- `substr(pos, len)` 的第二个参数是长度, 不是右端点。
- `find/rfind` 找不到返回 `string::npos`, 不是 `-1`; 用 `if (p != string::npos)` 判断。
- `string::npos` 类型是 `size_t`, 不要写 `int p = s.find(t)` 后再和 `-1` 混着判断。
- `size()` 是无符号类型; 倒序循环先写 `int n = (int)s.size()`。
- `insert/erase/replace` 都可能  $O(n)$ , 在循环中反复对长串中间操作会退化到  $O(n^2)$ 。
- 反复 `s = s + c` 可能复制多次; 追加单字符优先 `push_back`。
- `stoi/stoll` 要求字符串合法且范围足够; 不确定时手写扫描或用 `stringstream`。
- `isdigit/isalpha/tolower/toupper` 传入 `char` 前推荐转 `unsigned char`。
- `getline` 混 `cin >>` 时, 第一次 `getline` 前先 `cin.ignore(...)`。

暴力/部分替代:

- 查找子串小数据可直接双循环逐字符比较。
- 删除/替换小数据直接 `erase/replace` 模拟。
- 复杂行格式看不懂时, 整行 `getline` 后用 `stringstream` 或手扫字符。

升级方向:

- 单模式多次匹配: KMP/Z。
- 多模式匹配: Trie/AC 自动机。
- 子串相等、大量回文判断: Rolling Hash。
- 回文子串统计: 中心扩展或 STR-05 Manacher。

最小测试样例:

```
s = abracadabra
s.substr(0,4) = abra
s.find("ra") = 2
s.rfind("ra") = 9
"abc" < "abd" = true
split_by_char("a,b,,c", ',') = ["a", "b", "", "c"]
```

## STR-01 string 常用操作

模块编号: STR-01

模块名称: C++ string 常用操作与索引转换

标签: [字符串][string][基础操作][0-index]

一句话用途: 统一字符串读入、截取、查找、拼接、排序和 0-index/1-index 转换, 减少低级错误。

索引约定:

本模块内部使用 C++ string 自然 0-index: 位置  $0..n-1$ 。

题面若给第  $k$  个字符, 通常是 1-index: 读入后用  $k--$ 。

子串 `substr(pos, len)` 的 `pos` 是 0-index, `len` 是长度, 不是右端点。

若题面给闭区间  $[l, r]$  且为 1-index, 则 C++ 写 `s.substr(l-1, r-l+1)`。

题面触发词:

- 字符串处理、字符替换、统计字符。
- 子串、前缀、后缀。
- 字典序排序。
- 翻转、拼接、删除、插入。
- 大小写转换。

什么时候用:

- 题目只需要简单字符串操作, 不需要 KMP/Hash。
- 字符串长度不大, 可以直接用 `substr/find`。
- 需要把题面 1-index 位置转成 C++ 0-index。

不要什么时候用:

- $n, m$  很大且要重复匹配子串, `find/substr` 可能 TLE。
- 需要大量判断任意子串是否相等, 优先 Rolling Hash。

- 多模式串匹配, 优先 Trie/AC 自动机。

复杂度:

- `s.size()`:  $O(1)$ 。
- `s.substr(pos, len)`:  $O(len)$ 。
- `s.find(t)`: 通常可用但最坏不作为算法保证。
- 排序字符串数组:  $O(\text{总比较成本} * \log n)$ 。

数据范围参考:

- $|s| \leq 1e5$ : 一次线性扫描没问题。
- 重复 `substr` 复制总长度可能到  $O(n^2)$ , 要小心。

依赖的标准容器:

- `string`。
- `vector<int>` / `vector<string>`。
- 字符计数常用 `array<int, 26>` 或 `vector<int>(256)`。

输入如何整理:

```
string s;
cin >> s; // 无空格字符串
```

```
string line;
getline(cin, line); // 含空格整行, 注意先处理上一行换行
```

接口:

```
to0(pos1) -> 题面 1-index 转 0-index
substr_lidx(s, l, r) -> 题面 1-index 闭区间子串
count_lower(s) -> 统计小写字母
is_prefix(s, t) -> t 是否为 s 的前缀
is_suffix(s, t) -> t 是否为 s 的后缀
```

输出能力:

- 字符频次。
- 子串、前缀、后缀。
- 字典序比较。
- 简单模拟修改后的字符串。

下游可接:

- KMP/Z 函数。
- Trie。
- Rolling Hash。
- STR-05 Manacher。

可拼接模块:

- STR-02 KMP/Z。
- STR-03 Trie/Rolling Hash。
- STR-05 Manacher。
- DP LCS/编辑距离。

模板代码:

```
int to0(int pos1) {
    return pos1 - 1;
}

string substr_lidx(const string &s, int l, int r) {
    if (l < 1 || r < 1 || r > (int)s.size()) return "";
    return s.substr(l - 1, r - l + 1);
}

array<int, 26> count_lower(const string &s) {
    array<int, 26> cnt{};
    for (char c : s) {
        if ('a' <= c && c <= 'z') cnt[c - 'a']++;
    }
}
```

```

    }
    return cnt;
}

bool is_prefix(const string &s, const string &t) {
    if (t.size() > s.size()) return false;
    for (int i = 0; i < (int)t.size(); i++) {
        if (s[i] != t[i]) return false;
    }
    return true;
}

bool is_suffix(const string &s, const string &t) {
    int n = (int)s.size(), m = (int)t.size();
    if (m > n) return false;
    for (int i = 0; i < m; i++) {
        if (s[n - m + i] != t[i]) return false;
    }
    return true;
}

```

调用示例:

```

string s = "abcdef";
int l = 2, r = 4; // 题面 1-index
cout << substr_lidx(s, l, r) << "\n"; // bcd

auto cnt = count_lower(s);
cout << cnt['a' - 'a'] << "\n";

```

常见坑:

- `s[i]` 是 0-index。
- `substr(pos, len)` 第二个参数是长度, 不是右端点。
- `getline` 前如果刚用过 `cin >> x`, 要吃掉换行。
- `char` 可能是 signed, 做 ASCII 桶建议转 `unsigned char` 或用 `vector<int>(256)`。
- 循环写 `i < s.size() - 1` 时, 空串会让无符号减法出事; 先转 `int n=s.size()`。

暴力/部分分替代:

- 子串匹配小数据: 从每个位置开始逐字符比较。
- 子串相等小数据: 直接 `substr` 比较。
- 多次修改小数据: 直接改 `string`。

升级方向:

- 单模式匹配大数据 -> KMP/Z。
- 多模式前缀统计 -> Trie。
- 任意子串相等 -> Rolling Hash。
- 回文子串 -> STR-05 Manacher 或中心扩展。

最小测试样例:

```

s = abcdef
题面 [2,4] -> substr_lidx = bcd
is_prefix(abcdef, abc) = true
is_suffix(abcdef, def) = true

```

---

## CPP-013 STL 容器成员函数速查

模块编号: CPP-013

模块名称: STL 容器成员函数速查

标签: STL、vector、deque、list、array、string、stack、queue、priority\_queue、set、multiset、map、multimap、unordered\_map、unordered\_set、pair、tuple、iterator

一句话用途：在考场上快速确认 STL 容器该用哪个成员函数、复杂度大致是多少、哪些写法安全可抄。

题面触发词：数组、队列、栈、堆、集合、映射、去重、计数、前驱后继、哈希表、字符串、二元组、多元组、遍历删除。

什么时候用：已经知道算法方向，但卡在容器接口、删除写法、堆排序规则、哈希表预留空间或 pair/tuple 写法时。

不要什么时候用：需要自己实现线段树、树状数组、DSU、图算法主体时，本模块只提供 STL 接口速查，不替代算法模块。

复杂度：顺序容器按位置访问/插入删除不同；红黑树容器  $O(\log n)$ ；哈希容器平均  $O(1)$ ；堆 push/pop  $O(\log n)$ 、top  $O(1)$ 。

数据范围参考：n <= 2e5 时 STL 容器通常可用；哈希表大量插入前建议 reserve 和 max\_load\_factor；需要稳定最坏复杂度时优先有序容器。

依赖的标准容器：vector、deque、list、array、string、stack、queue、priority\_queue、set、multiset、map、multimap、unordered\_map、unordered\_set、pair、tuple。

输入如何整理：

- 连续下标、DP 表、邻接表：优先 vector。
- 两端进出、单调队列、0-1 BFS：优先 deque。
- 先进先出：queue；后进先出：stack。
- 每次取最大/最小：priority\_queue。
- 自动排序、去重、前驱后继：set/map。
- 只按 key 快速查找/计数：unordered\_map/unordered\_set。
- 多字段排序或存状态：pair/tuple，字段多且含义复杂时改 struct。

接口：

- 通用查询：c.empty()、c.size()。
- 端点访问：c.front()、c.back()；栈和堆用 c.top()。
- 清空：c.clear()；注意 array 没有 clear()。
- 改大小：vector/string/deque 常用 resize()；array 大小固定。
- 替换内容：vector/string/deque/list 可用 assign()。
- 预留容量：vector/string/unordered\_map/unordered\_set 常用 reserve()。

输出能力：输出容器元素、计数结果、映射值、堆顶、集合最小/最大值、排序后的记录字段。

下游可接：排序、二分、前缀和、BFS、Dijkstra、贪心、DP、离散化、扫描线、记忆化搜索。

可拼接模块：CPP-002 基础容器、CPP-003 排序二分、CPP-004 队列栈堆、CPP-005 关联容器、CPP-007 坐标压缩、CPP-009 常见 RE/WA。

## 顺序容器简表

| 容器          | 常用用途              | 常用接口                                                | 关键提醒                               |
|-------------|-------------------|-----------------------------------------------------|------------------------------------|
| vector<T>   | 数组、表、邻接表、DP       | push_back、pop_back、back、resize、reserve、assign、clear | 支持 a[i]；中间插删慢；扩容会使旧迭代器/引用/指针失效     |
| deque<T>    | 两端队列、单调队列、0-1 BFS | push_front、push_back、pop_front、pop_back、front、back  | 支持 dq[i]；两端快，中间插删仍不适合大量使用          |
| list<T>     | 已知迭代器位置的频繁插删      | push_back、push_front、insert、erase、sort、splice       | 不支持 a[i]；遍历只能用迭代器；算法题较少用           |
| array<T, N> | 固定小数组、方向数组        | fill、front、back、begin、end、size                      | 大小编译期固定；没有 push_back/clear/resize  |
| string      | 字符串、字符数组          | push_back、pop_back、substr、find、resize、reserve、clear | size() 是无符号类型；空串不能 s[0]、front、back |

## 通用成员函数速查

| 函数      | 作用   | 可抄写法                   | 注意                     |
|---------|------|------------------------|------------------------|
| empty() | 是否为空 | if (c.empty())         | 访问端点前先判空               |
| size()  | 元素个数 | int n = (int)c.size(); | 和 int 比较时常强转           |
| clear() | 清空元素 | c.clear();             | vector 容量通常不变；array 没有 |

| 函数            | 作用        | 可抄写法                                           | 注意                                             |
|---------------|-----------|------------------------------------------------|------------------------------------------------|
| front()       | 第一个元素     | int x = c.front();                             | vector/deque/list/array/string/queue 支持, 非空才可用 |
| back()        | 最后一个元素    | int x = c.back();                              | queue 也有 back()                                |
| top()         | 栈顶/堆顶     | int x = st.top();                              | 只用于 stack/priority_queue                       |
| resize(n)     | 改变元素个数    | a.resize(n + 1);                               | 变大补默认值, 变小删除尾部                                 |
| resize(n, v)  | 改大小并指定新增值 | a.resize(n, -1);                               | 只影响新增元素, 不会重置旧元素                               |
| reserve(n)    | 预留容量      | a.reserve(n);                                  | 不改变 size(), 不能直接访问新位置                          |
| assign(n, v)  | 替换为 n 个 v | a.assign(n + 1, 0);                            | 会清掉原内容                                         |
| begin()/end() | 半开区间迭代器   | for (auto it = c.begin(); it != c.end(); ++it) | end() 不是最后一个元素                                 |

## vector/deque/list/array/string 常用模板

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<int> a;
    a.reserve(n);           // 只预留容量, size 仍是 0
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        a.push_back(x);
    }

    a.resize(n + 1, 0);      // size 变为 n + 1, 新增位置补 0
    a.assign(n + 1, -1);    // 整个 vector 变成 n + 1 个 -1
    a.clear();              // size 变 0

    deque<int> dq;
    dq.push_back(1);
    dq.push_front(2);
    if (!dq.empty()) {
        cout << dq.front() << ' ' << dq.back() << '\n';
    }

    list<int> ls = {3, 1, 2};
    ls.sort();              // List 不能用 sort(ls.begin(), ls.end())

    array<int, 4> dir = {0, 1, 0, -1};
    dir.fill(0);

    string s = "abc";
    s.push_back('d');
    cout << s.substr(1, 2) << '\n'; // 从下标 1 开始取 2 个字符, 输出 bc

    return 0;
}
```

## stack/queue/priority\_queue

| 容器                      | 用途   | 插入      | 查看      | 删除    | 备注                 |
|-------------------------|------|---------|---------|-------|--------------------|
| stack<T>                | 后进先出 | push(x) | top()   | pop() | 括号、DFS 模拟、撤销       |
| queue<T>                | 先进先出 | push(x) | front() | pop() | BFS; 也可 back() 看队尾 |
| priority_queue<T> 默认最大堆 |      | push(x) | top()   | pop() | 每次取最大              |
| priority_queue<T, 最小堆   |      | push(x) | top()   | pop() | 每次取最小              |
| vector<T>, greater<T>>  |      |         |         |       |                    |

```

stack<int> st;
st.push(1);
if (!st.empty()) {
    int x = st.top();
    st.pop();
}

queue<int> q;
q.push(1);
if (!q.empty()) {
    int u = q.front();
    q.pop();
}

priority_queue<int> max_heap;
max_heap.push(5);
max_heap.push(2);
cout << max_heap.top() << '\n'; // 5

priority_queue<int, vector<int>, greater<int>> min_heap;
min_heap.push(5);
min_heap.push(2);
cout << min_heap.top() << '\n'; // 2

```

## priority\_queue 存 pair 与自定义排序

```

using pii = pair<int, int>;

// pair 默认字典序: 先比较 first, 再比较 second.
// 最小堆常用于 Dijkstra: {距离, 点}
priority_queue<pii, vector<pii>, greater<pii>> pq;
pq.push({0, 1});
while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
}

struct Node {
    int dist, id;
};

struct Cmp {
    bool operator()(const Node& a, const Node& b) const {
        if (a.dist != b.dist) return a.dist > b.dist; // dist 小的优先
        return a.id > b.id; // id 小的优先
    }
};

priority_queue<Node, vector<Node>, Cmp> heap;
heap.push({3, 2});
heap.push({1, 5});

```

## set/multiset

| 容器          | 特点         | 常用接口                                            | 适合场景            |
|-------------|------------|-------------------------------------------------|-----------------|
| set<T>      | 自动排序, 自动去重 | insert、erase、find、count、lower_bound、upper_bound | 去重、有序遍历、前驱后继    |
| multiset<T> | 自动排序, 允许重复 | 同 set                                           | 动态维护一堆数, 重复值要保留 |

```
set<int> s;
s.insert(3);
s.insert(1);

if (s.count(3)) {
    cout << "exist\n";
}

auto it = s.lower_bound(2);    // 第一个 >= 2
if (it != s.end()) cout << *it << '\n';

// 前驱: 严格小于 x 的最大值
int x = 3;
auto p = s.lower_bound(x);
if (p != s.begin()) {
    --p;
    cout << *p << '\n';
}

multiset<int> ms;
ms.insert(5);
ms.insert(5);

// multiset 只删除一个 5
auto one = ms.find(5);
if (one != ms.end()) {
    ms.erase(one);
}

// ms.erase(5) 会删除所有 5
```

## map/multimap

| 容器             | 特点           | 常用接口                                      | 适合场景               |
|----------------|--------------|-------------------------------------------|--------------------|
| map<K, V>      | key 自动排序且唯一  | mp[k]、insert、find、count、erase、lower_bound | 映射、计数、有序 key 查询    |
| multimap<K, V> | key 自动排序且可重复 | insert、equal_range、find、erase             | 一个 key 对应多条记录且需要有序 |

```
map<string, int> cnt;
cnt["alice"]++;

if (cnt.find("bob") == cnt.end()) {
    cout << "bob not found\n";
}

for (auto [key, value] : cnt) {
    cout << key << ' ' << value << '\n';
}

multimap<int, string> mm;
mm.insert({90, "alice"});
mm.insert({90, "bob"});
```

```

auto range = mm.equal_range(90);
for (auto it = range.first; it != range.second; ++it) {
    cout << it->first << ' ' << it->second << '\n';
}

```

## unordered\_map/unordered\_set

| 容器                  | 特点          | 常用接口                                            | 适合场景      |
|---------------------|-------------|-------------------------------------------------|-----------|
| unordered_set<T>    | 哈希集合, 不保证顺序 | insert、erase、find、count、reserve、max_load_factor | 快速判重、存在性  |
| unordered_map<K, V> | 哈希映射, 不保证顺序 | mp[k]、find、count、erase、reserve、max_load_factor  | 快速计数、快速查值 |

```

int n;
cin >> n;

unordered_map<long long, int> cnt;
cnt.max_load_factor(0.7);
cnt.reserve(n * 2 + 1);

for (int i = 1; i <= n; i++) {
    long long x;
    cin >> x;
    cnt[x]++;
}

unordered_set<int> seen;
seen.max_load_factor(0.7);
seen.reserve(n * 2 + 1);

seen.insert(10);
if (seen.find(10) != seen.end()) {
    cout << "seen\n";
}

```

## pair/tuple

```

using pii = pair<int, int>;
using tiii = tuple<int, int, int>;

pii p = {3, 5};
cout << p.first << ' ' << p.second << '\n';

auto [x, y] = p;

tuple<int, int, long long> state = {1, 2, 100LL};
auto [i, j, val] = state;

vector<pair<int, int>> v = {{2, 3}, {1, 9}, {1, 4}};
sort(v.begin(), v.end()); // 默认先 first 升序, 再 second 升序

// 自定义排序: first 升序; first 相同时 second 降序
sort(v.begin(), v.end(), [](const pii& a, const pii& b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second;
});

```

## 迭代器遍历与 erase 安全写法

```

// vector/string/deque/list/set/map/unordered_* 通用: erase 返回下一个合法迭代器
for (auto it = c.begin(); it != c.end(); ) {
    if (need_delete(*it)) {

```



```

        it = c.erase(it);
    } else {
        ++it;
    }
}

// map/unordered_map 删除时, 元素是 pair<const K, V>
for (auto it = mp.begin(); it != mp.end(); ) {
    if (it->second == 0) {
        it = mp.erase(it);
    } else {
        ++it;
    }
}

// 不要在 range-for 中直接 erase 当前容器
// 错误示例:
// for (int x : v) {
//     if (x < 0) v.erase(...);
// }

```

```

// vector 按条件删除的可靠写法
v.erase(remove(v.begin(), v.end(), value), v.end());

v.erase(remove_if(v.begin(), v.end(), [](int x) {
    return x < 0;
}), v.end());

```

## 常用完整模板

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int, int>;
const int MAXN = 200000 + 5;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    static int sorted[MAXN];

    unordered_map<int, int> cnt;
    cnt.max_load_factor(0.7);
    cnt.reserve(n * 2 + 1);

    priority_queue<pii, vector<pii>, greater<pii>> min_heap;
    set<int> unique_values;

    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        sorted[i] = x;
        cnt[x]++;
        unique_values.insert(x);
        min_heap.push({x, i});
    }

    sort(sorted + 1, sorted + n + 1);

```

```

while (!min_heap.empty()) {
    auto [value, id] = min_heap.top();
    min_heap.pop();
    cout << value << ' ' << id << '\n';
}

return 0;
}

```

调用示例:

```

// 1-index 数组: 上限明确时优先静态数组
static int a[MAXN];
for (int i = 1; i <= n; i++) cin >> a[i];

// 邻接表: 上限明确时直接静态数组, 每个点一个 vector
static vector<int> g[MAXN];
g[u].push_back(v);

// 固定方向数组
array<int, 4> dx = {-1, 0, 1, 0};
array<int, 4> dy = {0, 1, 0, -1};

// map 的 lower_bound: 第一个 key >= x
auto it = mp.lower_bound(x);
if (it != mp.end()) {
    cout << it->first << ' ' << it->second << '\n';
}

```

常见坑:

- front/back/top 前必须先确认非空。
- reserve(n) 只改容量, 不改元素个数; 写完 reserve 后不能用 a[i] = x 填新元素。
- resize(n, v) 只给新增位置填 v, 旧位置不会被重置。
- assign(n, v) 会替换整个容器内容, 旧内容全部消失。
- clear() 后 size() 为 0, 但 vector/string 的容量通常还在。
- array 大小固定, 没有 push\_back、pop\_back、clear、resize。
- priority\_queue 默认最大堆; 小根堆写 priority\_queue<T, vector<T>, greater<T>>。
- pair 默认排序是字典序, 先 first 后 second。
- set/map 的 lower\_bound 是按有序 key 查; unordered\_map/unordered\_set 没有 lower\_bound。
- unordered\_map/unordered\_set 大量插入前, 推荐先 max\_load\_factor(0.7) 再 reserve(n \* 2 + 1)。
- mp[key] 会在 key 不存在时创建默认值; 只判断存在用 find 或 count。
- multiset.erase(x) 删除所有等于 x 的元素; 只删一个要先 find, 存在再 erase(it)。
- 遍历时删除元素要写 it = c.erase(it), 不要在 range-for 中删除当前容器。
- string::find() 找不到时返回 string::npos, 不要拿它和 -1 混用。
- list 不支持随机访问, 也不能用普通 sort(begin, end), 要用成员函数 ls.sort()。

暴力/部分替代: 小数据可以用 vector 存全部元素, 每次线性扫描、排序或手动删除; 确认思路后再替换成 set/map/unordered\_map/priority\_queue。

升级方向: 哈希计数接记忆化搜索; set/map 接扫描线和离线查询; priority\_queue 接 Dijkstra、贪心合并; vector 接前缀和、树状数组、线段树和 DP。

最小测试样例:

```

输入
3
5 2 5

输出
2 2
5 1
5 3

```

# CPP-012 STL 常用算法速查

模块编号: CPP-012

模块名称: 常用 STL 算法速查模块

标签: C++17、STL、algorithm、numeric、排序、二分、去重、排列、前缀和、考场速查

一句话用途: 把常用 STL 算法按“考场能直接抄”的方式集中列出, 快速拿排序、二分、统计、去重、排列和前缀处理的基础分。

题面触发词: 排序、稳定排序、去重、删除所有、查找第一个、区间计数、是否存在、最大最小、第  $k$  小、下一个排列、旋转数组、填充、编号、前缀和、总和、gcd、lcm。

什么时候用: 数据已经放进 vector/string/array, 需要对一段连续区间做排序、查找、统计、批量赋值或数值累加时。

不要什么时候用: 需要频繁动态插入删除并保持有序时, 优先用 set/multiset/map; 需要复杂区间修改查询时, 优先用树状数组、Segment Tree 或 Sparse Table。

复杂度:

- 排序类: sort/stable\_sort/nth\_element 通常  $O(n \log n)$ , 其中 nth\_element 平均  $O(n)$ 。
- 二分类: lower\_bound/upper\_bound/binary\_search/equal\_range 为  $O(\log n)$ , 前提是区间已按同一规则有序。
- 线性扫描类: reverse/unique/erase-remove/rotate/fill/iota/accumulate/partial\_sum/count/find/count\_if/find\_if/all\_of 为  $O(n)$ 。
- gcd/lcm 为  $O(\log \min(a,b))$ 。

数据范围参考:

- $n \leq 2e5$ : 排序、线性扫描、前缀和都很稳。
- $n \leq 1e6$ : 排序仍常见, 注意常数、内存和多测清空。
- $q$  很大且每次问区间数量: 先排序, 再用二分, 避免每次线性扫。

依赖的标准容器:

- vector、array、string: 连续区间算法最常用。
- deque: 也支持随机访问迭代器, 可用于 sort, 但算法题通常转 vector 更直观。
- set/map 等有序容器有自己的 lower\_bound, 不要把全局 lower\_bound 当成  $O(\log n)$  用在普通双向迭代器上。

接口:

```

sort(begin, end)
lower_bound(begin, end, x)
v.erase(unique(begin, end), end)
v.erase(remove(begin, end, x), end)
iota(begin, end, start)
accumulate(begin, end, 0LL)
partial_sum(begin, end, output_begin)
```

依赖头文件:

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
```

## 1. 考场总表

| 算法           | 常用写法                                            | 用途             | 坑点              |
|--------------|-------------------------------------------------|----------------|-----------------|
| sort         | sort(v.begin(), v.end())                        | 升序排序           | 会打乱原顺序          |
| stable_sort  | stable_sort(v.begin(), v.end(), cmp)            | 相等元素保持原相对顺序    | 比 sort 常数大      |
| reverse      | reverse(v.begin(), v.end())                     | 整段反转           | 区间仍是半开 $[l,r)$  |
| unique       | v.erase(unique(v.begin(), v.end()), v.end())    | 删除相邻重复         | 要“值去重” 通常先排序    |
| erase-remove | v.erase(remove(v.begin(), v.end(), x), v.end()) | 删除所有等于 $x$ 的元素 | remove 不会真的缩短容器 |
| lower_bound  | lower_bound(v.begin(), v.end(), x)              | 第一个 $\geq x$   | 必须有序            |

| 算法               | 常用写法                                           | 用途                                | 坑点                       |
|------------------|------------------------------------------------|-----------------------------------|--------------------------|
| upper_bound      | upper_bound(v.begin(), v.end(), x)             | 第一个 > x                           | 必须有序                     |
| binary_search    | binary_search(v.begin(), v.end(), x)           | 判断 x 是否存在                         | 只返回 bool                 |
| equal_range      | equal_range(v.begin(), v.end(), x)             | 一次拿 [第一个 >=x, 第一个 >x)             | 计数用 second - first       |
| min/max          | min(a,b), max(a,b)                             | 两个值取较小/较大                         | 类型尽量一致                   |
| minmax_element   | minmax_element(v.begin(), v.end())             | 一次找最小和最大元素位置                      | 空区间不能解引用                 |
| nth_element      | nth_element(v.begin(), v.begin()+k, v.end())   | 把 0-index 下标 k 的元素放到位, 也就是第 k+1 小 | 题目问第 k 小时通常写 begin()+k-1 |
| next_permutation | next_permutation(v.begin(), v.end())           | 下一个字典序排列                          | 枚举全排列前先排序                |
| prev_permutation | prev_permutation(v.begin(), v.end())           | 上一个字典序排列                          | 降序起点才能枚举全部逆向排列           |
| rotate           | rotate(v.begin(), v.begin()+k, v.end())        | 把中点搬到开头                           | k 是迭代器位置, 不是次数本身         |
| fill             | fill(v.begin(), v.end(), val)                  | 批量赋值                              | 多维数组更推荐循环逐行填             |
| iota             | iota(v.begin(), v.end(), start)                | 生成连续编号                            | 头文件在 numeric, bits 已含    |
| accumulate       | accumulate(v.begin(), v.end(), 0LL)            | 求和或折叠                             | 初值决定返回类型                 |
| partial_sum      | partial_sum(v.begin(), v.end(), pre.begin()+1) | 前缀和                               | 目标空间要提前开够                |
| gcd              | gcd(a,b)                                       | 最大公约数                             | C++17 在 <numeric>        |
| lcm              | lcm(a,b)                                       | 最小公倍数                             | 可能溢出                     |
| count            | count(v.begin(), v.end(), x)                   | 统计等于 x 的数量                        | 线性复杂度                    |
| find             | find(v.begin(), v.end(), x)                    | 找第一个等于 x 的位置                      | 找不到返回 end()              |
| count_if         | count_if(v.begin(), v.end(), pred)             | 统计满足条件的数量                         | pred 返回 bool             |
| find_if          | find_if(v.begin(), v.end(), pred)              | 找第一个满足条件的位置                       | 找不到返回 end()              |
| all_of           | all_of(v.begin(), v.end(), pred)               | 是否全部满足                            | 空区间返回 true               |
| any_of           | any_of(v.begin(), v.end(), pred)               | 是否存在一个满足                          | 空区间返回 false              |
| none_of          | none_of(v.begin(), v.end(), pred)              | 是否没有元素满足                          | 空区间返回 true               |

## 2. 排序、比较函数与 lambda 正确写法

考场优先记住一句：比较函数 cmp(a,b) 表示 “a 是否应该排在 b 前面”，相等时必须返回 false。

```

struct Node {
    int id;
    int score;
    int age;
};

vector<Node> a;

// 分数高在前; 分数相同, 年龄小在前; 仍相同, 编号小在前。
sort(a.begin(), a.end(), [](const Node &x, const Node &y) {
    if (x.score != y.score) return x.score > y.score;
    if (x.age != y.age) return x.age < y.age;
    return x.id < y.id;
});

```

错误写法:

```
// 错：相等时也可能返回 true，破坏严格弱序。
sort(a.begin(), a.end(), [](const Node &x, const Node &y) {
    return x.score >= y.score;
});

常用排序抄法：

sort(v.begin(), v.end());           // 升序
sort(v.rbegin(), v.rend());         // 降序，适合 int/LL/string/pair
stable_sort(v.begin(), v.end(), cmp); // 相等元素保留原相对顺序

pair/tuple 默认按字典序排序：

vector<pair<int, int>> p;
sort(p.begin(), p.end()); // 先按 first，再按 second，都是升序
```

### 3. 半开区间与闭区间换算

STL 统一使用半开区间  $[first, last)$ ：包含左端点，不包含右端点。

1-index 数组闭区间  $[L, R]$  换成 STL 迭代器：

```
// a[0] 不用，处理 a[L] 到 a[R]
sort(a.begin() + L, a.begin() + R + 1);
reverse(a.begin() + L, a.begin() + R + 1);
fill(a.begin() + L, a.begin() + R + 1, 0);
```

静态数组也可以直接用指针：

```
sort(a + L, a + R + 1);
```

半开区间  $[l, r)$  的长度永远是：

```
int len = r - l;
```

二分返回下标：

```
int pos = (int)(lower_bound(v.begin(), v.end(), x) - v.begin());
```

### 4. 去重与删除

值去重标准三连：

```
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
```

只删除相邻重复，不排序：

```
v.erase(unique(v.begin(), v.end()), v.end());
```

删除所有等于  $x$  的元素：

```
v.erase(remove(v.begin(), v.end(), x), v.end());
```

按条件删除：

```
v.erase(remove_if(v.begin(), v.end(), [](int x) {
    return x < 0;
}), v.end());
```

考场口令：unique/remove/remove\_if 都只是把“保留的元素”搬到前面并返回新尾巴，真正缩短 vector 要再接 erase。

### 5. 二分查找与区间计数

前提：v 已经升序排序。

```
sort(v.begin(), v.end());
```

```
int x;
cin >> x;
```

```
auto it1 = lower_bound(v.begin(), v.end(), x); // 第一个 >= x
auto it2 = upper_bound(v.begin(), v.end(), x); // 第一个 > x
```

```
bool ok = binary_search(v.begin(), v.end(), x);
int cnt_x = (int)(it2 - it1);
```

统计闭区间  $[L, R]$  内有多少个数:

```
int cnt = (int)(upper_bound(v.begin(), v.end(), R)
    - lower_bound(v.begin(), v.end(), L));
```

equal\_range 一次拿到等值范围:

```
auto range = equal_range(v.begin(), v.end(), x);
int cnt = (int)(range.second - range.first);
```

找最后一个  $\leq x$  的位置:

```
auto it = upper_bound(v.begin(), v.end(), x);
if (it == v.begin()) {
    // 不存在  $\leq x$  的元素
} else {
    --it;
    int pos = (int)(it - v.begin());
}
```

降序数组二分容易写错。考场建议: 能升序就升序; 如果必须降序, 排序和二分要使用同一个比较规则。

```
sort(v.begin(), v.end(), greater<int>());
auto it = lower_bound(v.begin(), v.end(), x, greater<int>());
```

## 6. 最大最小、第 $k$ 小与重排

两个值取最小最大:

```
int lo = min(a, b);
int hi = max(a, b);
ll best = min({x, y, z});
```

区间一次找最小最大元素位置:

```
auto [mn_it, mx_it] = minmax_element(v.begin(), v.end());
if (mn_it != v.end()) {
    int mn = *mn_it;
    int mx = *mx_it;
}
```

第  $k$  小, 题面  $k$  通常按 1-index 理解:

```
//  $a[1..n]$ , 第  $k$  小会被放到  $a[k]$ 
nth_element(a + 1, a + k, a + n + 1);
int kth = a[k];
```

拿最小的  $k$  个数, 但这  $k$  个内部不要求有序:

```
nth_element(a + 1, a + k, a + n + 1);
//  $a[1..k]$  是  $k$  个较小元素, 但内部乱序。
```

如果还要输出有序结果, 再补一刀:

```
sort(a + 1, a + k + 1);
```

reverse 和 rotate:

```
reverse(v.begin(), v.end());
```

```
int k = 3;
rotate(v.begin(), v.begin() + k, v.end()); //  $[0, k)$  搬到末尾, 原  $k$  位置变开头
```

右旋  $k$  位:

```
int n = (int)v.size();
if (n > 0) {
    k %= n;
    rotate(v.begin(), v.end() - k, v.end());
}
```

## 7. 排列枚举

从小到大枚举所有不同排列:

```
sort(v.begin(), v.end());
do {
    // 使用当前排列
} while (next_permutation(v.begin(), v.end()));
```

从大到小枚举:

```
sort(v.rbegin(), v.rend());
do {
    // 使用当前排列
} while (prev_permutation(v.begin(), v.end()));
```

考场提醒: 全排列是阶乘复杂度,  $n > 10$  通常只能拿部分分或需要换思路。

## 8. 批量赋值、编号、求和与前缀和

批量赋值:

```
fill(v.begin(), v.end(), 0);
```

二维 vector 清空:

```
for (auto &row : dp) fill(row.begin(), row.end(), INF);
```

生成连续编号:

```
static int id[MAXN];
iota(id + 1, id + n + 1, 1); // 1, 2, ..., n
```

求和必须注意初值类型:

```
ll sum = accumulate(v.begin(), v.end(), 0LL);
```

前缀和,  $pre[0]=0$ ,  $pre[i]$  表示  $a[1..i]$  之和:

```
static ll a[MAXN], pre[MAXN];
partial_sum(a + 1, a + n + 1, pre + 1);
```

// 1-index 闭区间  $[L, R]$  的和

```
ll ans = pre[R] - pre[L - 1];
```

如果原数组是 int 且和可能超过 int, 考场更稳的写法是手写 long long 前缀:

```
static ll pre[MAXN];
for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i];
```

自定义累加:

```
ll total_abs = accumulate(v.begin(), v.end(), 0LL, [](ll s, int x) {
    return s + llabs((ll)x);
});
```

## 9. gcd 与 lcm

C++17 可直接用:

```
ll g = gcd(a, b);
ll l = lcm(a, b);
```

防溢出版 lcm 更适合考场:

```
ll lcm_limit(ll a, ll b, ll limit) {
    if (a == 0 || b == 0) return 0;
    __int128 aa = a, bb = b;
    if (aa < 0) aa = -aa;
    if (bb < 0) bb = -bb;
    ll g = gcd(a, b);
    aa /= g;
    __int128 lim = limit;
    ll over = (limit == LLONG_MAX ? limit : limit + 1);
```

```

    if (bb != 0 && aa > lim / bb) return over;
    __int128 res = aa * bb;
    if (res > lim) return over;
    return (ll)res;
}

```

多个数合并:

```

ll g = 0;
for (ll x : v) g = gcd(g, x);

ll l = 1;
for (ll x : v) l = lcm_limit(l, x, 4'000'000'000'000'000'000LL);

```

## 10. 统计、查找与判定

线性统计:

```

int c = (int)count(v.begin(), v.end(), x);
int odd = (int)count_if(v.begin(), v.end(), [](int x) {
    return x % 2 != 0;
});

```

线性查找:

```

auto it = find(v.begin(), v.end(), x);
if (it != v.end()) {
    int pos = (int)(it - v.begin());
}

auto first_big = find_if(v.begin(), v.end(), [](int x) {
    return x > 100;
});

```

全部满足:

```

bool all_pos = all_of(v.begin(), v.end(), [](int x) {
    return x > 0;
});

```

存在满足:

```

bool has_even = any_of(v.begin(), v.end(), [](int x) {
    return x % 2 == 0;
});

bool no_negative = none_of(v.begin(), v.end(), [](int x) {
    return x < 0;
});

```

考场选择:

- 无序数组找一次: find/count/all\_of/any\_of, 简单稳。
- 有序数组查很多次: lower\_bound/upper\_bound/binary\_search/equal\_range。
- 需要频繁查存在性且不关心顺序: unordered\_set。

## 11. 综合模板

```

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 200000 + 5;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;

```



```

cin >> n >> q;

static ll a[MAXN], sorted[MAXN], pre[MAXN];
for (int i = 1; i <= n; i++) cin >> a[i];

for (int i = 1; i <= n; i++) sorted[i] = a[i];
sort(sorted + 1, sorted + n + 1);

partial_sum(a + 1, a + n + 1, pre + 1);

while (q--) {
    ll L, R;
    cin >> L >> R;

    int cnt_value_range =
        (int)(upper_bound(sorted + 1, sorted + n + 1, R)
            - lower_bound(sorted + 1, sorted + n + 1, L));

    cout << cnt_value_range << '\n';
}

return 0;
}

```

调用示例:

```

int v[6] = {0, 3, 1, 2, 2, 5};
int n = 5;

sort(v + 1, v + n + 1); // 1 2 2 3 5
int cnt2 = (int)(upper_bound(v + 1, v + n + 1, 2)
    - lower_bound(v + 1, v + n + 1, 2)); // 2

n = unique(v + 1, v + n + 1) - (v + 1); // v[1..n] = 1 2 3 5

bool has3 = binary_search(v + 1, v + n + 1, 3); // true
ll sum = accumulate(v + 1, v + n + 1, 0LL); // 11

```

常见坑:

- STL 区间是半开 [first,last); 闭区间 [L,R] 要写到 begin()+R+1。
- lower\_bound/upper\_bound/binary\_search/equal\_range 只能用于已按同一规则排序的区间。
- 自定义 cmp 不能写 <= 或 >=, 相等必须返回 false。
- sort 不稳定; 相等元素要保留原相对顺序时用 stable\_sort。
- unique 只删除相邻重复; 值去重前先 sort。
- remove/remove\_if 不会改变 vector 长度, 必须配合 erase。
- find/lower\_bound/minmax\_element 返回迭代器, 解引用前检查是否为 end()。
- accumulate(v.begin(), v.end(), 0) 会按 int 累加, 容易溢出; 写 0LL。
- partial\_sum 的累加类型来自输入元素, vector<int> 求大前缀和时优先手写 long long 循环。
- nth\_element 只能保证第 k 小到位, 不能保证整段有序。
- next\_permutation 要从升序开始才会枚举所有排列。
- lcm 可能溢出, 涉及上限判断时用 a / gcd(a,b) \* b 并先检查乘法。
- 空区间下 all\_of 返回 true, any\_of 返回 false, 不要被边界样例骗。

暴力/部分替代:

- 小数据查询区间数量: 每次线性扫 O(nq)。
- 小数据找第 k 小: 每次完整 sort。
- 小数据判断存在: 直接 find, 不用先建复杂结构。

升级方向:

- 大量动态插入删除 + 有序查询: multiset 或平衡树思路。
- 大量区间和/区间修改: 树状数组或 Segment Tree。
- 大量区间最值静态查询: Sparse Table。
- 排序后还要映射回原值范围: 接坐标压缩模块。

最小测试样例:

```
v = {3, 1, 2, 2, 5}
sort -> {1, 2, 2, 3, 5}
unique after erase -> {1, 2, 3, 5}
count of [2,3] by bounds -> 2
accumulate with 0LL -> 11
gcd(12,18) -> 6
lcm(12,18) -> 36
```

---

## CPP-003 sort/lambda/比较函数/lower\_bound/upper\_bound

模块编号: CPP-003

模块名称: 排序、lambda、自定义比较与二分位置

标签: sort、stable\_sort、lambda、比较函数、lower\_bound、upper\_bound、离线排序

一句话用途: 处理“排序后扫描”“按多个关键字排序”“在有序数组里找位置/计数”。

题面触发词: 从小到大、从大到小、排名、第一个不小于、最后一个不大于、区间内有多少数、按分数排序、离线处理。

什么时候用: 需要重排数组、按字段优先级排序、在有序 vector 中查找边界时。

不要什么时候用: 数据会频繁动态插入删除并保持有序时, 用 set/multiset/map; 数组未排序时不能直接二分。

复杂度: 排序  $O(n \log n)$ ; lower\_bound/upper\_bound 每次  $O(\log n)$ 。

数据范围参考:  $n \leq 2e5$  排序很常见;  $n \leq 1e6$  也通常可接受, 但注意常数和内存。

依赖的标准容器: vector、pair、tuple、自定义 struct。

输入如何整理: 先把需要排序/二分的值放入 vector; 需要保留原位置就存 (值, 原编号)。

接口:

- `sort(v.begin(), v.end())`: 升序。
- `sort(v.rbegin(), v.rend())`: 降序, 适合基础类型。
- `sort(v.begin(), v.end(), cmp)`: 自定义比较。
- `lower_bound(v.begin(), v.end(), x)`: 第一个  $\geq x$ 。
- `upper_bound(v.begin(), v.end(), x)`: 第一个  $> x$ 。

输出能力: 可输出排序后的顺序、排名、满足区间  $[L, R]$  的数量或位置。

下游可接: 坐标压缩、贪心、离线查询、双指针、Kruskal、扫描线。

可拼接模块: CPP-002 基础容器、CPP-005 关联容器、CPP-007 坐标压缩。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Node {
    int id;
    ll score;
    int age;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<Node> a(n + 1);
    vector<ll> xs;
```

```

for (int i = 1; i <= n; i++) {
    cin >> a[i].score >> a[i].age;
    a[i].id = i;
    xs.push_back(a[i].score);
}

sort(a.begin() + 1, a.end(), [](const Node &x, const Node &y) {
    if (x.score != y.score) return x.score > y.score; // 分数高在前
    if (x.age != y.age) return x.age < y.age; // 年龄小在前
    return x.id < y.id; // 编号小在前
});

sort(xs.begin(), xs.end());

ll L, R;
cin >> L >> R;
int left = (int)(lower_bound(xs.begin(), xs.end(), L) - xs.begin());
int right = (int)(upper_bound(xs.begin(), xs.end(), R) - xs.begin());
int count_in_range = right - left;

if (n == 0) return 0;
cout << a[1].id << '\n';
cout << count_in_range << '\n';

return 0;
}

```

调用示例:

```

vector<int> v = {1, 2, 2, 4, 7};
int x = 2;
int first_ge = (int)(lower_bound(v.begin(), v.end(), x) - v.begin()); // 1
int first_gt = (int)(upper_bound(v.begin(), v.end(), x) - v.begin()); // 3
int count_x = first_gt - first_ge; // 2

```

常见坑:

- 二分前必须保证 vector 已经按同一种规则排序。
- 自定义比较必须写严格顺序: 相等时返回 false, 不要写 <=。
- 降序数组不能直接用默认 lower\_bound, 除非额外传同样的比较规则; 考场上更推荐升序二分。
- lower\_bound 返回的是迭代器, 转下标要减 begin()。
- 返回 v.end() 表示没找到可用位置, 不能直接解引用。
- 排序会打乱原顺序, 需要原编号时先存 id。

暴力/部分替代: 小数据可每次线性扫描找第一个满足条件的位置, 复杂度  $O(nq)$ 。

升级方向: 大量区间计数接坐标压缩 + 树状数组; 动态有序接 set/multiset/map。

最小测试样例:

输入

```

3
90 18
90 17
80 20
85 100

```

输出

```

2
2

```

---

## CPP-004 queue/deque/stack/priority\_queue

模块编号: CPP-004

模块名称：队列、双端队列、栈与优先队列

标签：queue、deque、stack、priority\_queue、BFS、单调队列、堆

一句话用途：处理先进先出、后进先出、两端进出和“每次取最大/最小”的场景。

题面触发词：BFS、层数、最短步数、撤销、括号匹配、滑动窗口、每次取最大、每次取最小、合并代价、Dijkstra。

什么时候用：需要固定顺序弹出元素，或需要反复取得当前最大/最小元素时。

不要什么时候用：需要按任意值删除堆中元素时，普通 priority\_queue 不支持；需要有序遍历所有元素时用 set/map。

复杂度：queue/deque/stack 常用操作  $O(1)$ ；priority\_queue 插入/弹出  $O(\log n)$ ，取堆顶  $O(1)$ 。

数据范围参考：BFS 状态数  $\leq 1e6$  常见；堆操作  $\leq 2e5$  到  $1e6$  通常可用。

依赖的标准容器：queue、deque、stack、priority\_queue、vector、pair。

输入如何整理：图/状态转移整理成可扩展的邻接表或候选状态；堆元素常存（距离，点）、（权值，编号）。

接口：

- q.push(x) / q.front() / q.pop() / q.empty()。
- dq.push\_front(x) / dq.push\_back(x) / dq.front() / dq.back()。
- st.push(x) / st.top() / st.pop()。
- 最大堆：priority\_queue<int> pq。
- 最小堆：priority\_queue<T, vector<T>, greater<T>> pq。

输出能力：输出 BFS 距离、处理顺序、当前最大/最小值、括号是否合法等。

下游可接：图论 BFS/Dijkstra、单调队列优化 DP、贪心、模拟。

可拼接模块：CPP-002 基础容器、CPP-003 排序二分、CPP-008 整数溢出。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    priority_queue<int> max_heap;
    priority_queue<int, vector<int>, greater<int>> min_heap;

    int n;
    cin >> n;
    if (n <= 0) return 0;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        max_heap.push(x);
        min_heap.push(x);
    }

    cout << max_heap.top() << ' ' << min_heap.top() << '\n';

    queue<int> q;
    vector<int> dist(n + 1, -1);
    q.push(1);
    dist[1] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        int v = u + 1;
        if (v <= n && dist[v] == -1) {
            dist[v] = dist[u] + 1;
        }
    }
}
```

```

        q.push(v);
    }
}

cout << dist[n] << '\n';

return 0;
}

调用示例:

// Dijkstra 常用最小堆元素: {当前距离, 点}
priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
pq.push({0, 1});
while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
}

// 栈: 括号匹配
stack<char> st;
bool ok = true;
for (char c : s) {
    if (c == '(') st.push(c);
    else if (c == ')') {
        if (st.empty()) ok = false;
        else st.pop();
    }
}
bool balanced = ok && st.empty();

```

常见坑:

- front/top/back 前必须确认容器非空, 否则容易 RE。
- priority\_queue 默认是最大堆。
- 最小堆写法里第二个模板参数必须是底层容器 vector<T>。
- 堆里旧状态不会自动删除, Dijkstra 常用 if (d != dist[u]) continue; 跳过旧元素。
- stack 只能看栈顶, 不能遍历; 需要遍历用 vector。
- deque 两端操作快, 中间插入删除仍不适合大量使用。

暴力/部分替代: 数据小可以每次线性扫描找最小/最大, 复杂度  $O(n^2)$ ; 能过小数据后再换堆。

升级方向: BFS 接无权最短路; 最小堆接 Dijkstra; deque 接 0-1 BFS 或单调队列优化。

最小测试样例:

输入  
5  
3 1 4 1 5

输出  
5 1  
4

---

## CPP-005 set/multiset/map/unordered\_map

模块编号: CPP-005

模块名称: 有序集合、可重集合、映射和哈希映射

标签: set、multiset、map、unordered\_map、计数、去重、有序查找

一句话用途: 处理去重、计数、动态有序查找、键值映射和快速按键访问。

题面触发词: 去重、出现次数、字典、映射、动态插入删除、第一个不小于、前驱后继、按名字统计、按值分组。

什么时候用: 需要自动排序、自动去重、保存键到值的关系, 或需要快速判断某个键是否出现时。

不要什么时候用：只需要连续下标数组时不要用 map；需要区间和/排名且数据很大时考虑坐标压缩 + 树状数组。

复杂度：set/multiset/map 插入删除查找  $O(\log n)$ ；unordered\_map 平均  $O(1)$ ，最坏情况可能退化。

数据范围参考： $2e5$  级别动态有序操作用红黑树容器稳；键是字符串或大整数且只需查值时可用哈希映射。

依赖的标准容器：set、multiset、map、unordered\_map、string、vector。

输入如何整理：把需要查重/计数的值作为 key；若 key 是大坐标且后续要接数组结构，优先压缩。

接口：

- s.insert(x)：插入。
- s.erase(x)：删除所有等于 x 的元素，set 中最多一个。
- auto it = ms.find(x); if (it != ms.end()) ms.erase(it);: multiset 只删一个 x。
- s.lower\_bound(x)：第一个  $\geq x$ 。
- mp[key]++：计数，但会创建默认值。
- mp.count(key)：判断 key 是否存在。

输出能力：输出去重后的有序序列、某值出现次数、某键对应答案、前驱后继。

下游可接：离散化、扫描线、贪心、模拟、记忆化搜索。

可拼接模块：CPP-003 排序二分、CPP-007 坐标压缩、CPP-009 语言坑。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    set<int> unique_values;
    multiset<int> bag;
    map<string, int> name_count;
    unordered_map<int, int> fast_count;
    fast_count.reserve(n * 2 + 1);
    fast_count.max_load_factor(0.7);

    for (int i = 1; i <= n; i++) {
        int x;
        string name;
        cin >> x >> name;

        unique_values.insert(x);
        bag.insert(x);
        name_count[name]++;
        fast_count[x]++;
    }

    int query_x;
    cin >> query_x;
    auto it = unique_values.lower_bound(query_x);

    cout << (int)unique_values.size() << '\n';
    if (it == unique_values.end()) cout << -1 << '\n';
    else cout << *it << '\n';
    auto it_cnt = fast_count.find(query_x);
    cout << (it_cnt == fast_count.end() ? 0 : it_cnt->second) << '\n';

    return 0;
}
```

调用示例：

```

// multiset 只删除一个 x
auto it = ms.find(x);
if (it != ms.end()) {
    ms.erase(it);
}

// map 按 key 从小到大遍历
for (auto [key, value] : mp) {
    cout << key << ' ' << value << '\n';
}

// set 前驱: 严格小于 x 的最大值
auto it = s.lower_bound(x);
if (it != s.begin()) {
    --it;
    int predecessor = *it;
}

```

常见坑:

- `mp[x]` 会在 `x` 不存在时创建键; 只判断存在性用 `count` 或 `find`。
- `multiset.erase(x)` 会删除所有 `x`; 只删一个要先 `find`, 存在再 `erase(it)`。
- `unordered_map` 没有顺序, 不能找前驱后继, 也不能 `lower_bound`。
- `set` 中元素不能直接修改; 要先删除旧值, 再插入新值。
- 迭代器到 `begin()` 时不能再 `--it`。
- 需要第 `k` 小、动态排名时, 普通 STL 不直接支持, 常转坐标压缩 + 树状数组。

暴力/部分替代: 小数据用 `vector` 存所有元素, 每次排序/扫描查找, 复杂度较高但容易写。

升级方向: 值域大且要排名时接 Compressor + 树状数组; 复杂状态缓存接 `map<tuple<...>, value>`。

最小测试样例:

输入

```

4
5 a
2 b
5 a
8 c
4

```

输出

```

3
5
0

```

## CPP-006 bitset 与位运算

模块编号: CPP-006

模块名称: bitset 与整数位运算

标签: bitset、mask、位运算、子集、状态压缩、快速集合

一句话用途: 用二进制位表示集合、状态和可达性, 快速做包含、枚举、计数和转移。

题面触发词: 子集、状态压缩、选或不选、集合、二进制、开关、可达和、最多 20 个点、位标记。

什么时候用: 元素数较小可用 `mask` 表示集合; 可达性长度固定且不太大时用 `bitset` 加速。

不要什么时候用: 位数运行时才知道且很大时, `bitset<N>` 不方便;  $n > 25$  的全子集枚举通常爆炸。

复杂度: 单个整数位操作  $O(1)$ ; 枚举所有子集  $O(2^n)$ ; `bitset` 按机器字批量运算, 常数很小。

数据范围参考:  $n \leq 20$  常可枚举所有 `mask`;  $n \leq 60$  可用 `long long` 存集合; 可达和  $\leq 1e5$  常用 `bitset`。

依赖的标准容器: `bitset`、`vector`、整数类型 `int` / `long long`。

输入如何整理: 题面第  $i$  个元素仍编号  $1..n$ , 映射到 `bit i-1`; 字符串/开关可转成 `mask`; 可达和用 `bitset<MAXS + 1>`。

接口:

- `mask & (1u << (i - 1))`: 测试题面第  $i$  个元素, 适合 `unsigned int mask`。
- `mask | (1u << (i - 1))`: 加入题面第  $i$  个元素。
- `mask & ~(1u << (i - 1))`: 删除题面第  $i$  个元素。
- `1LL << (i - 1)`:  $i$  可能超过 31 时用 `long long mask`。
- `__builtin_popcount(mask)`: 统计 `int` 的 1 个数。
- `__builtin_popcountll(mask)`: 统计 `long long` 的 1 个数。
- `bs.test(i) / bs.set(i) / bs.reset(i)`: `bitset` 单点操作。

输出能力: 输出集合大小、状态编号、是否可达、满足条件的子集数量。

下游可接: 暴力枚举、状压 DP、背包可达性、图上小点集搜索。

可拼接模块: C++-002 基础容器、C++-008 整数溢出、DP 状压模块。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

const int MAXS = 100000;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, target;
    cin >> n >> target;

    bitset<MAXS + 1> can;
    can[0] = 1;

    int mask = 0;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        if (0 <= x && x <= MAXS) can |= (can << x);
        if (i <= 30 && (x % 2 != 0)) mask |= (1 << (i - 1));
    }

    cout << (0 <= target && target <= MAXS && can[target] ? 1 : 0) << '\n';
    cout << __builtin_popcount((unsigned)mask) << '\n';

    return 0;
}
```

调用示例:

```
// 枚举 mask 的所有非空子集
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // sub 是 mask 的一个非空子集
}

// 遍历题面元素编号 1..n
for (int i = 1; i <= n; i++) {
    if (mask & (1 << (i - 1))) {
        // 第 i 个元素被选中
    }
}

// Long Long 位移, k 可能 >= 31 时必须写 1LL
long long one = 1LL << k;
```

常见坑:

- `1 << 40` 会溢出 `int`, 要写 `1LL << 40`。
- `__builtin_ctz(0)`、`__builtin_ctzll(0)` 不合法, 调用前先判断非零。



- `bitset<N>` 的 `N` 必须是编译期常量。
- 子集枚举是指数复杂度，看到 `n=30` 以上要谨慎。
- `mask` 位号自然从 0 开始；题面对象编号仍保持 `1..n`。`int mask` 写 `1u << (i - 1)`；`long long mask` 写 `1LL << (i - 1)`。
- `~mask` 会把高位也变成 1，清位时写 `mask & ~(1 << i)`，并确保类型合适。

暴力/部分替代：元素少时直接 DFS 枚举选/不选；可达和也可用 `vector<int>` 做普通背包。

升级方向：子集枚举接状压 DP；`bitset` 接背包可达性、集合交并、字符串匹配加速。

最小测试样例：

输入

```
3 5
2 3 4
```

输出

```
1
1
```

## CPP-007 坐标压缩 Compressor

模块编号：CPP-007

模块名称：坐标压缩

标签：坐标压缩、离散化、`lower_bound`、`upper_bound`、1-index 编号

一句话用途：把很大的值域压成 `1..k`，方便接 树状数组、线段树、差分和数组。

题面触发词：坐标很大、值域到 `1e9/1e18`、只出现少量点、离散化、排名、逆序对、区间覆盖、扫描线。

什么时候用：值很大但不同值数量不多，并且需要用数组下标维护这些值时。

不要什么时候用：原值之间的距离有实际长度意义且需要保留每段长度时，必须额外保存相邻原坐标差；不能只看压缩编号差。

复杂度：建表排序  $O(n \log n)$ ；每次取编号  $O(\log n)$ 。

数据范围参考：原坐标可到 `1e18`；不同坐标数 `k` 通常  $\leq 2e5$  时适合压缩后接数组结构。

依赖的标准容器：`vector<ll>`、`sort`、`unique`、`lower_bound`、`upper_bound`。

输入如何整理：先收集所有会用到的坐标和值；如果是区间覆盖题，通常要收集端点，必要时也收集 `r + 1`。

接口：

- `build(v)`：用所有候选坐标建表。
- `id(x)`：`x` 确定出现过时取 1-index 编号。
- `lower_id(x)`：第一个原值  $\geq x$  的编号。
- `upper_id(x)`：最后一个原值  $\leq x$  的编号。
- `val(pos)`：压缩编号还原成原值。
- `size()`：压缩后坐标数量。

输出能力：输出压缩编号、排名、区间 `[L, R]` 对应的压缩下标范围。

下游可接：树状数组、线段树、差分、扫描线、逆序对、动态排名。

可拼接模块：CPP-003 排序二分、CPP-008 整数溢出、数据结构卷 树状数组/SegmentTree。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct Compressor {
    vector<ll> xs;

    void build(vector<ll> v) {
        xs = v;
        sort(xs.begin(), xs.end());
```

```

        xs.erase(unique(xs.begin(), xs.end()), xs.end());
    }

    int id(ll x) const {
        return (int)(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1;
    }

    int lower_id(ll x) const {
        return (int)(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1;
    }

    int upper_id(ll x) const {
        return (int)(upper_bound(xs.begin(), xs.end(), x) - xs.begin());
    }

    ll val(int pos) const {
        return xs[pos - 1];
    }

    int size() const {
        return (int)xs.size();
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<ll> a(n + 1), all;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        all.push_back(a[i]);
    }

    Compressor comp;
    comp.build(all);

    for (int i = 1; i <= n; i++) {
        cout << comp.id(a[i]) << (i == n ? '\n' : ' ');
    }

    ll L, R;
    cin >> L >> R;
    int l = comp.lower_id(L);
    int r = comp.upper_id(R);
    cout << (l <= r ? r - l + 1 : 0) << '\n';

    return 0;
}

```

调用示例:

```

Compressor comp;
comp.build(all_values);

int pos = comp.id(x);           // x 出现过
int l = comp.lower_id(L);       // 第一个 >= L
int r = comp.upper_id(R);       // 最后一个 <= R
if (l <= r) {
    // 压缩区间 [L, r] 非空
}

```

常见坑:

- 只压缩出现过的点, 压缩编号差不等于原坐标距离。
- 查询  $[L, R]$  时,  $L/R$  不一定出现过; 要用 `lower_id/upper_id`。
- 压缩后本卷约定从 1 开始, 方便直接接 树状数组/SegmentTree。
- 区间覆盖如果需要半开边界, 常要加入  $r + 1$ ; 当  $r$  很大时注意溢出。
- 忘记 `unique` 前先 `sort` 会去重失败。
- `id(x)` 默认  $x$  在表中; 不确定时先判断 `lower_bound` 结果。

暴力/部分替代: 值域小可以直接开数组; 数据小可以用 `map<ll, int>` 动态分配编号。

升级方向: 接树状数组做逆序对/排名; 接线段树做区间维护; 接扫描线做矩形/区间统计。

最小测试样例:

输入

```
5
100 50 100 1000000000 50
60 1000000000
```

输出

```
2 1 2 3 1
2
```

---

## C++-008 long long 与 \_\_int128 溢出处理

模块编号: C++-008

模块名称: 整数类型、乘法溢出与大整数中间量

标签: long long、\_\_int128、溢出、取模、二分中点、距离无穷大

一句话用途: 避免答案、乘法、距离和计数在中间过程爆掉导致 WA。

题面触发词: 答案很大、方案数、权值到  $1e9$ 、乘积、平方、最短路距离、取模、二分答案、 $1e18$ 。

什么时候用: 权值/答案/计数可能超过 `int`, 或两个 `long long` 相乘可能超过  $9e18$  时。

不要什么时候用: 真正需要上百位整数时, C++ 内建类型不够, 要按题意使用字符串高精或大整数思路。

复杂度: 类型转换本身  $O(1)$ ; `__int128` 比普通整数慢一点, 但考场中用于中间量很稳。

数据范围参考: `int` 约到  $2e9$ ; `long long` 约到  $9e18$ ;  $1e9 * 1e9$  必须先转 `long long`;  $1e18 * 1e18$  需要 `__int128` 中间量。

依赖的标准容器: 无; 常与 `vector<ll>`、图论距离数组、DP 数组拼接。

输入如何整理: 点数、下标用 `int`; 权值、距离、答案、方案数优先读入 `long long`; 乘法比较时转 `__int128`。

接口:

- `using ll = long long`: 常用整数答案类型。
- `using i128 = __int128_t`: 大乘法中间量。
- `print_i128(x)`: 输出 `__int128`。
- $l + (r - l) / 2$ : 安全二分中点。
- `LINF = 4'000'000'000'000'000'000LL`: 距离无穷大哨兵。

输出能力: 输出 `long long` 答案或 `__int128` 答案。

下游可接: 最短路、DP、组合计数、二分答案、数学取模。

可拼接模块: C++-001 主骨架、C++-003 二分、图论 Dijkstra、数学快速幂。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using i128 = __int128_t;

const ll LINF = 4'000'000'000'000'000'000LL;
```

```

void print_i128(i128 x) {
    if (x == 0) {
        cout << 0;
        return;
    }
    if (x < 0) {
        cout << '-';
        x = -x;
    }
    string s;
    while (x > 0) {
        int digit = (int)(x % 10);
        s.push_back((char)('0' + digit));
        x /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ll a, b, limit;
    cin >> a >> b >> limit;

    i128 product = (i128)a * b;
    cout << (product <= (i128)limit ? "OK" : "OVER") << '\n';

    print_i128(product);
    cout << '\n';

    ll l = 0, r = limit;
    ll mid = l + (r - l) / 2;
    cout << mid << '\n';

    return 0;
}

```

调用示例:

// 先转再乘

```
long long area = 1LL * width * height;
```

// 比较  $a * b \leq c$ , 避免  $a*b$  溢出

```

if ((__int128)a * b <= c) {
    // safe
}

```

// 最短路松弛

```

if (dist[u] != LINF && dist[v] > dist[u] + w) {
    dist[v] = dist[u] + w;
}

```

常见坑:

- `int * int` 会先按 `int` 乘完再赋给 `long long`, 必须写 `1LL * a * b`。
- `abs(x)` 遇到 `long long` 时注意函数重载, 稳妥写 `llabs(x)` 或自己处理。
- 二分中点写  $(l + r) / 2$  可能溢出, 稳妥写  $l + (r - l) / 2$ 。
- `LINF + w` 可能溢出, 最短路松弛前先判断 `dist[u] != LINF`。
- `__int128` 不能直接 `cin/cout`, 需要自己读写或只作中间比较。
- 取模乘法时, 两个数都可能到  $1e18$ , 中间乘法用 `__int128` 再 `% mod`。

暴力/部分替代: 小数据可先用 `long long`, 但一旦样例外数据范围接近  $1e18$ , 必须处理乘法中间量。

升级方向: 接快速幂、组合计数、二分答案、几何面积、最短路距离。

最小测试样例：

输入

1000000000000 1000000000000 1000000000000000000

输出

OVER

1000000000000000000000000000000

5000000000000000000

---

## CPP-009 常见 RE/WA 语言坑清单

模块编号：CPP-009

模块名称：常见 RE/WA 语言坑清单

标签：RE、WA、越界、空容器、迭代器、初始化、多组数据、比较函数

一句话用途：提交前按清单扫一遍，优先排除语言层面的低级错误。

题面触发词：多组数据、边界、空集合、没有答案、字符串、动态删除、排序、哈希、递归。

什么时候用：代码写完后、样例过了但担心隐藏点时；出现 RE/WA/TLE 不知道从哪查时。

不要什么时候用：算法逻辑明显不对时不要只改语言细节；先确认模型、复杂度和边界。

复杂度：检查清单本身无复杂度；防御性判断通常  $O(1)$ 。

数据范围参考：数组大小接近上限、多组数据总量大、递归深度大、答案接近  $1e18$  时尤其要查。

依赖的标准容器：vector、string、queue、stack、set、map、unordered\_map。

输入如何整理：每组数据都重新初始化容器；读字符串行前处理换行；数组按统一 1-index 开  $n + 1$ 。

接口：

- empty()：访问队首、栈顶、末尾前先判断。
- assign(size, value)：多组数据重置 vector。
- clear()：清空容器。
- find()：删除或解引用迭代器前先确认存在。
- count()：只判断是否存在。

输出能力：帮助定位错误类型，不直接产生题目答案。

下游可接：所有模块的提交前检查。

可拼接模块：CPP-001 主骨架、CPP-002 基础容器、CPP-005 关联容器、CPP-008 整数溢出。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;

        vector<ll> a;
        a.assign(n + 1, 0);
        for (int i = 1; i <= n; i++) cin >> a[i];

        stack<int> st;
```

```

    if (!st.empty()) {
        cout << st.top() << '\n';
    }

    set<int> s;
    s.insert(1);
    auto it = s.find(1);
    if (it != s.end()) {
        s.erase(it);
    }

    cout << a[n] << '\n';
}

return 0;
}

```

调用示例:

```

// 边遍历边删除 map/set 的安全写法
for (auto it = s.begin(); it != s.end(); ) {
    if (need_delete(*it)) {
        it = s.erase(it);
    } else {
        ++it;
    }
}

// 多组数据重置邻接表
g.assign(n + 1, {});
dist.assign(n + 1, -1);

```

常见坑:

- 越界: 1-index 数组要开  $n + 1$ , 循环别写到  $n + 1$ 。
- 空容器: front/back/top 前先判断 empty()。
- 字符串: cin >> s 不读空格, getline 前注意行尾换行。
- 多组数据: vector/map/set/queue 没清空会串数据。
- 比较函数: 排序比较不能用  $\leq$ , 相等时必须返回 false。
- 迭代器: 删除后旧迭代器失效; 按容器返回的新迭代器继续。
- map[key]: 会创建新键; 只查存在用 find/count。
- 哈希容器: 遍历顺序不固定, 不能依赖输出顺序。
- 整数: 乘法先转 long long 或 \_\_int128; 答案变量不要用 int。
- 递归: 深 DFS 可能栈溢出; 可改显式栈或确认深度安全。
- 浮点: 比较小数不要直接用  $==$ , 一般用误差。
- 下标混用: 数组/图默认 1-index, 字符串默认 0-index, 写转换时标注清楚。

暴力/部分分替代: 遇到 RE 先把可疑访问加边界判断; 遇到 WA 先打印或手算极小样例验证索引。

升级方向: 把本清单放到提交前流程; 质检时可自动扫描大数组、空容器访问和禁止写法。

最小测试样例:

```

输入
1
3
10 20 30

输出
30

```

## v0.2 本卷例题训练区

这一节是 0.2 新增的实战例题。每题都配完整可运行代码和样例; 考试时优先看“覆盖模块”和“考场用途”, 再复制对应代码骨架。

## V01-EX01 班级成绩汇总

- 归属卷: 第 1 卷
- 覆盖模块: CPP-001 主骨架与输入输出、CPP-002 vector、CPP-10 格式化输出
- 考场用途: 练习 cin/cout 快速骨架、1-index vector 存数、fixed << setprecision 输出平均值。

**题目描述:** 给定一个班级 n 名同学的整数成绩, 输出总分、平均分和最高分。

**输入格式:** 第一行一个整数 n。第二行 n 个整数, 表示每名同学的成绩。

**输出格式:** 输出三行, 分别为 sum= 总分、average= 平均分、max= 最高分。平均分保留两位小数。

**样例输入:**

```
5
80 90 75 100 95
```

**样例输出:**

```
sum=440
average=88.00
max=100
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> score(n + 1);
    long long sum = 0;
    int best = 0;
    for (int i = 1; i <= n; i++) {
        cin >> score[i];
        sum += score[i];
        best = max(best, score[i]);
    }

    double avg = 1.0 * sum / n;
    cout << "sum=" << sum << '\n';
    cout << fixed << setprecision(2) << "average=" << avg << '\n';
    cout << "max=" << best << '\n';
    return 0;
}
```

**测试设计:**

1. 输入:

```
3
1 2 3
```

期望输出:

```
sum=6
average=2.00
max=3
```

2. 输入:

```
1
99
```

期望输出:

```
sum=99
average=99.00
max=99
```

## V01-EX02 商品小票打印

- 归属卷：第 1 卷
- 覆盖模块：C++-10 scanf/printf、格式化输出、补零、小数位
- 考场用途：练习只使用 scanf/printf 的传统 IO，避免关闭同步后混用 C 和 C++ 输入输出。

**题目描述：**给定  $n$  件商品的编号、单价和数量，输出每件商品的小计和总价。商品编号按 4 位补零输出，小计和总价保留两位小数。

**输入格式：**第一行一个整数  $n$ 。接下来  $n$  行，每行包含整数  $id$ 、实数  $price$ 、整数  $count$ 。

**输出格式：**前  $n$  行输出 编号 小计，最后一行输出 TOTAL 总价。

**样例输入：**

```
3
7 12.5 2
42 3.2 5
105 100 1
```

**样例输出：**

```
0007 25.00
0042 16.00
0105 100.00
TOTAL 141.00
```

**完整代码：**

```
#include <bits/stdc++.h>
using namespace std;

struct Item {
    int id;
    double price;
    int count;
};

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    vector<Item> item(n + 1);
    double total = 0.0;
    for (int i = 1; i <= n; i++) {
        if (scanf("%d%lf%d", &item[i].id, &item[i].price, &item[i].count) != 3) return 0;
    }

    for (int i = 1; i <= n; i++) {
        double subtotal = item[i].price * item[i].count;
        total += subtotal;
        printf("%04d %.2f\n", item[i].id, subtotal);
    }
    printf("TOTAL %.2f\n", total);
    return 0;
}
```

**测试设计：**

1. 输入：

```
2
1 1.5 3
23 10 2
```

期望输出：



```
0001 4.50
0023 20.00
TOTAL 24.50
```

2. 输入:

```
1
9999 0.99 10
```

期望输出:

```
9999 9.90
TOTAL 9.90
```

### V01-EX03 EOF 单词频率表

- 归属卷: 第 1 卷
- 覆盖模块: CPP-001 EOF、CPP-10 getline、CPP-011 string、CPP-005 map
- 考场用途: 练习 getline 读到 EOF、stringstream 拆词、map 自动按字典序输出。

**题目描述:** 输入若干行文本, 统计每个单词出现次数。单词只按空白分隔, 大小写视为不同单词。

**输入格式:** 若干行文本, 直到文件结束。

**输出格式:** 按单词字典序输出, 每行一个单词和出现次数。

**样例输入:**

```
apple banana apple
pear banana
```

**样例输出:**

```
apple 2
banana 2
pear 1
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    map<string, int> cnt;
    string line;
    while (getline(cin, line)) {
        stringstream ss(line);
        string word;
        while (ss >> word) {
            cnt[word]++;
        }
    }

    for (const auto &kv : cnt) {
        cout << kv.first << ' ' << kv.second << '\n';
    }
    return 0;
}
```

**测试设计:**

1. 输入:

```
one
one two
```

期望输出:

one 2

two 1

2. 输入:

z y x

期望输出:

x 1

y 1

z 1

#### V01-EX04 通讯录号码查询

- 归属卷: 第 1 卷
- 覆盖模块: C++-005 unordered\_map、C++-011 string、C++-001 cin/cout
- 考场用途: 练习用 unordered\_map 做姓名到号码的快速查询, 并在大量插入前预留空间。

**题目描述:** 给定通讯录中若干人的姓名和号码, 再回答若干次查询。若姓名存在, 输出号码; 否则输出 NOT FOUND。

**输入格式:** 第一行一个整数  $n$ 。接下来  $n$  行, 每行一个姓名和号码。再输入整数  $q$ , 接下来  $q$  行每行一个待查询姓名。

**输出格式:** 对每次查询输出一行结果。

**样例输入:**

```
3
Alice 10086
Bob 10010
Cindy 95588
4
Bob
David
Alice
Cindy
```

**样例输出:**

```
10010
NOT FOUND
10086
95588
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    unordered_map<string, string> phone;
    phone.reserve(n * 2 + 10);
    phone.max_load_factor(0.7);

    for (int i = 1; i <= n; i++) {
        string name, number;
        cin >> name >> number;
        phone[name] = number;
    }

    int q;
    cin >> q;
    for (int i = 1; i <= q; i++) {
        string name;
        cin >> name;
```

```

        auto it = phone.find(name);
        if (it == phone.end()) {
            cout << "NOT FOUND\n";
        } else {
            cout << it->second << '\n';
        }
    }
    return 0;
}

```

#### 测试设计:

##### 1. 输入:

```

1
Tom 123
3
Tom
Jerry
Tom

```

##### 期望输出:

```

123
NOT FOUND
123

```

##### 2. 输入:

```

2
A 1
B 2
2
B
A

```

##### 期望输出:

```

2
1

```

### V01-EX05 任务优先级调度

- 归属卷: 第 1 卷
- 覆盖模块: CPP-004 priority\_queue、CPP-002 vector/string、结构体比较
- 考场用途: 练习 priority\_queue 默认最大堆思想: 优先级越大越先处理, 优先级相同则耗时短者先处理, 再按输入顺序稳定打破平局。

**题目描述:** 给定  $n$  个任务, 每个任务有名称、优先级和耗时。按规则输出处理顺序: 优先级高的先处理; 优先级相同, 耗时短的先处理; 仍相同, 先输入的先处理。

**输入格式:** 第一行一个整数  $n$ 。接下来  $n$  行, 每行包含任务名 name、整数优先级 priority、整数耗时 time。

**输出格式:** 输出  $n$  行, 每行一个任务的名称、优先级和耗时。

#### 样例输入:

```

5
write 2 30
fix 5 10
test 5 5
read 1 1
pack 2 20

```

#### 样例输出:

```

test 5 5
fix 5 10
pack 2 20
write 2 30
read 1 1

```

#### 完整代码:

```

#include <bits/stdc++.h>
using namespace std;

struct Task {
    string name;
    int priority;
    int time;
    int id;

    bool operator<(const Task &other) const {
        if (priority != other.priority) return priority < other.priority;
        if (time != other.time) return time > other.time;
        return id > other.id;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    priority_queue<Task> pq;
    for (int i = 1; i <= n; i++) {
        Task t;
        cin >> t.name >> t.priority >> t.time;
        t.id = i;
        pq.push(t);
    }

    while (!pq.empty()) {
        Task t = pq.top();
        pq.pop();
        cout << t.name << ' ' << t.priority << ' ' << t.time << '\n';
    }
    return 0;
}

```

#### 测试设计:

##### 1. 输入:

```

3
a 1 10
b 3 20
c 2 5

```

##### 期望输出:

```

b 3 20
c 2 5
a 1 10

```

##### 2. 输入:

```

3
first 2 8
second 2 8
third 2 3

```

##### 期望输出:

```

third 2 3
first 2 8
second 2 8

```

## V01-EX06 整数去重排序

- 归属卷: 第 1 卷
- 覆盖模块: CPP-005 set、CPP-001 cin/cout
- 考场用途: 练习 set 自动去重和升序输出, 适合数据量中等、需要有序唯一集合的题。

**题目描述:** 给定 n 个整数, 输出不同整数的个数, 并按升序输出所有不同整数。

**输入格式:** 第一行一个整数 n。第二行 n 个整数。

**输出格式:** 第一行输出 count= 不同整数个数。第二行按升序输出不同整数, 用一个空格分隔。

**样例输入:**

```
8
5 3 5 2 3 9 2 1
```

**样例输出:**

```
count=5
1 2 3 5 9
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    set<int> values;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        values.insert(x);
    }

    cout << "count=" << values.size() << '\n';
    bool first = true;
    for (int x : values) {
        if (!first) cout << ' ';
        first = false;
        cout << x;
    }
    cout << '\n';
    return 0;
}
```

**测试设计:**

1. 输入:

```
5
4 4 4 4 4
```

期望输出:

```
count=1
4
```

2. 输入:

```
6
-1 3 -1 0 3 2
```

期望输出:

```
count=4
-1 0 2 3
```

## V01-EX07 排序后找第一个不小于目标的位置

- 归属卷：第 1 卷
- 覆盖模块：CPP-003 sort/lower\_bound、CPP-012 STL 算法、CPP-002 vector
- 考场用途：练习半开区间二分，输出排序后 1-index 位置，避免把迭代器差值当成原数组下标。

**题目描述：**给定  $n$  个整数和  $q$  次查询。先将数组升序排序。每次查询给定  $x$ ，找到排序后第一个大于等于  $x$  的数，输出它的位置和值；若不存在，输出  $-1$ 。

**输入格式：**第一行一个整数  $n$ 。第二行  $n$  个整数。第三行一个整数  $q$ 。接下来  $q$  行每行一个整数  $x$ 。

**输出格式：**每次查询输出一行。若找到，输出 位置 值；否则输出  $-1$ 。

**样例输入：**

```
6
8 1 5 3 5 10
4
0
5
6
11
```

**样例输出：**

```
1 1
3 5
5 8
-1
```

**完整代码：**

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    sort(a.begin() + 1, a.end());

    int q;
    cin >> q;
    for (int i = 1; i <= q; i++) {
        int x;
        cin >> x;
        auto it = lower_bound(a.begin() + 1, a.end(), x);
        if (it == a.end()) {
            cout << -1 << '\n';
        } else {
            int pos = (int)(it - a.begin());
            cout << pos << ' ' << *it << '\n';
        }
    }
    return 0;
}
```

**测试设计：**

1. 输入：

```
4
4 2 8 6
3
```

```
1
6
9
```

期望输出:

```
1 2
3 6
-1
```

2. 输入:

```
5
5 5 5 5 5
2
5
6
```

期望输出:

```
1 5
-1
```

### V01-EX08 学生成绩排行

- 归属卷: 第 1 卷
- 覆盖模块: C++-002 vector/string、C++-003 sort 与 lambda、C++-012 自定义比较
- 考场用途: 练习结构体数组的 1-index 存储和多关键字排序: 分数降序, 姓名升序。

**题目描述:** 给定  $n$  名学生的姓名和成绩, 按成绩从高到低排序; 成绩相同按姓名字典序升序排序。输出排名、姓名和成绩。

**输入格式:** 第一行一个整数  $n$ 。接下来  $n$  行, 每行一个姓名和一个整数成绩。

**输出格式:** 输出  $n$  行, 每行 排名 姓名 成绩。

**样例输入:**

```
5
Tom 90
Amy 95
Bob 90
Cindy 100
Dave 95
```

**样例输出:**

```
1 Cindy 100
2 Amy 95
3 Dave 95
4 Bob 90
5 Tom 90
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;

struct Student {
    string name;
    int score;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Student> a(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i].name >> a[i].score;
    }
}
```

```

    }

    sort(a.begin() + 1, a.end(), [](const Student &lhs, const Student &rhs) {
        if (lhs.score != rhs.score) return lhs.score > rhs.score;
        return lhs.name < rhs.name;
    });

    for (int i = 1; i <= n; i++) {
        cout << i << ' ' << a[i].name << ' ' << a[i].score << '\n';
    }
    return 0;
}

```

#### 测试设计:

1. 输入:

```

3
B 80
A 80
C 90

```

期望输出:

```

1 C 90
2 A 80
3 B 80

```

2. 输入:

```

2
Li 100
Wang 99

```

期望输出:

```

1 Li 100
2 Wang 99

```

#### V01-EX09 整行记录统计

- 归属卷: 第 1 卷
- 覆盖模块: CPP-10 getline、CPP-011 string、CPP-001 token 输入后处理换行
- 考场用途: 练习 `cin >> n` 后立刻 `getline` 前必须清掉行尾换行, 适合题目含空格字符串的场景。

**题目描述:** 给定  $n$  行记录。对每一行, 输出行号、字符长度和单词数量。单词按空白分隔。

**输入格式:** 第一行一个整数  $n$ 。接下来  $n$  行, 每行是一条记录, 可能包含空格。

**输出格式:** 输出  $n$  行, 每行 行号 长度 单词数量。

**样例输入:**

```

3
Hello World
abc 123
A B C

```

**样例输出:**

```

1 11 2
2 7 2
3 5 3

```

**完整代码:**

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

```



```

int n;
cin >> n;
cin.ignore(numeric_limits<streamsize>::max(), '\n');

vector<string> line(n + 1);
for (int i = 1; i <= n; i++) {
    getline(cin, line[i]);
}

for (int i = 1; i <= n; i++) {
    stringstream ss(line[i]);
    string word;
    int words = 0;
    while (ss >> word) words++;
    cout << i << ' ' << line[i].size() << ' ' << words << '\n';
}
return 0;
}

```

#### 测试设计:

1. 输入:

```

2
single
two words

```

期望输出:

```

1 6 1
2 9 2

```

2. 输入:

```

1
lead and tail

```

期望输出:

```

1 17 3

```

#### V01-EX10 闭区间计数查询

- 归属卷: 第 1 卷
- 覆盖模块: C++-003 lower\_bound/upper\_bound、C++-012 sort 与二分、C++-002 vector
- 考场用途: 练习在有序数组上统计  $[L, R]$  中元素个数, 避免手写二分边界出错。

**题目描述:** 给定  $n$  个整数和  $q$  个查询。每次查询给出  $L, R$ , 输出数组中落在闭区间  $[L, R]$  内的元素个数。

**输入格式:** 第一行一个整数  $n$ 。第二行  $n$  个整数。第三行一个整数  $q$ 。接下来  $q$  行每行两个整数  $L, R$ 。

**输出格式:** 对每个查询输出一行答案。

**样例输入:**

```

7
1 5 2 2 8 10 5
3
1 2
3 7
6 10

```

**样例输出:**

```

3
2
2

```

**完整代码:**

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    sort(a.begin() + 1, a.end());

    int q;
    cin >> q;
    for (int i = 1; i <= q; i++) {
        int l, r;
        cin >> l >> r;
        auto left_it = lower_bound(a.begin() + 1, a.end(), l);
        auto right_it = upper_bound(a.begin() + 1, a.end(), r);
        cout << (right_it - left_it) << '\n';
    }
    return 0;
}

```

#### 测试设计:

1. 输入:

```

5
1 2 3 4 5
2
2 4
6 9

```

期望输出:

```

3
0

```

2. 输入:

```

4
7 7 7 7
2
7 7
1 6

```

期望输出:

```

4
0

```

## 第 9 卷: Python 互补卷例题

### V01-CEX01 多行键值记录合并

- 归属卷: 第 1 卷
- 覆盖模块: getline、stringstream、map
- 考场用途: 处理含空格记录并按字典序输出。
- 参考题型来源: 参考来源: 洛谷入门字符串/模拟题型。

**题目描述:** 每行由若干 名字 数值 对组成, 合并所有名字的数值。

**输入格式:** 第一行  $n$ , 接下来  $n$  行记录。

**输出格式:** 按名字字典序输出合计。

样例输入:

```
3
alice 3 bob 2
alice 4
carl 5 bob 1
```

样例输出:

```
alice 7
bob 3
carl 5
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    map<string, long long> sum;
    for (int i = 1; i <= n; i++) {
        string line;
        getline(cin, line);
        stringstream ss(line);
        string key;
        long long value;
        while (ss >> key >> value) sum[key] += value;
    }
    for (auto [k, v] : sum) cout << k << ' ' << v << '\n';
    return 0;
}
```

测试设计: 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

---

## V01-CEX02 固定宽度成绩表

- 归属卷: 第 1 卷
- 覆盖模块: iomanip、fixed、setprecision、setw
- 考场用途: 复杂格式输出时直接套。
- 参考题型来源: 参考来源: 洛谷入门格式化输出题型。

题目描述: 输入若干学生两项成绩, 左对齐姓名、右对齐总分并保留两位小数。

输入格式: 第一行  $n$ , 之后  $\text{name a b}$ 。

输出格式: 每行输出宽度固定的姓名和总分。

样例输入:

```
2
Li 90 5.5
Wang 80.25 10
```

样例输出:

```
Li          95.50
Wang        90.25
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    cout << fixed << setprecision(2);
    for (int i = 1; i <= n; i++) {
        string name;
        double a, b;
        cin >> name >> a >> b;
        cout << setw(10) << left << name << right << setw(8) << a + b << '\n';
    }
    return 0;
}

```

**测试设计：**额外测试：构造最小规模、重复值、边界值各一组，和样例一起运行。

---

### V01-CEX03 哈希表加堆 TopK

- 归属卷：第 1 卷
- 覆盖模块：unordered\_map、priority\_queue
- 考场用途：词频统计后取最高频。
- 参考题型来源：参考来源：洛谷/ICPC 常见词频 TopK 模拟。

**题目描述：**统计字符串出现次数，输出出现次数最高的前 k 个；次数相同按字典序大的先出。

**输入格式：**第一行 n k，之后 n 个字符串。

**输出格式：**输出前 k 项。

**样例输入：**

```

7 2
a b a c b a c

```

**样例输出：**

```

a 3
c 2

```

**完整代码：**

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    unordered_map<string, int> cnt;
    cnt.reserve(n * 2 + 10);
    for (int i = 1; i <= n; i++) {
        string s;
        cin >> s;
        cnt[s]++;
    }
    priority_queue<pair<int, string>> pq;
    for (auto [s, c] : cnt) pq.push({c, s});
    for (int i = 1; i <= k && !pq.empty(); i++) {
        auto [c, s] = pq.top(); pq.pop();
        cout << s << ' ' << c << '\n';
    }
    return 0;
}

```

**测试设计:** 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

---

### V01-CEX04 pair 排序合并区间

- 归属卷: 第 1 卷
- 覆盖模块: vector、sort
- 考场用途: 用 STL 排序把模拟题变成标准区间合并。
- 参考题型来源: 参考来源: 洛谷区间合并/模拟题型。

**题目描述:** 给出若干闭区间, 合并相交区间并输出。

**输入格式:** 第一行  $n$ , 之后  $n$  行  $l$   $r$ 。

**输出格式:** 输出合并后区间个数和区间。

**样例输入:**

```
4
1 3
2 5
8 9
5 7
```

**样例输出:**

```
2
1 7
8 9
```

**完整代码:**

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<int,int>> seg(n + 1);
    for (int i = 1; i <= n; i++) cin >> seg[i].first >> seg[i].second;
    sort(seg.begin() + 1, seg.end());
    vector<pair<int,int>> ans;
    for (int i = 1; i <= n; i++) {
        if (ans.empty() || seg[i].first > ans.back().second) ans.push_back(seg[i]);
        else ans.back().second = max(ans.back().second, seg[i].second);
    }
    cout << ans.size() << '\n';
    for (auto [l, r] : ans) cout << l << ' ' << r << '\n';
    return 0;
}
```

**测试设计:** 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

---

### V01-CEX05 排序后二分查询

- 归属卷: 第 1 卷
- 覆盖模块: sort、lower\_bound、vector 1-index
- 考场用途: 二分函数的标准调用。
- 参考题型来源: 参考来源: 洛谷二分查找题型。

**题目描述:** 每次询问输出数组中第一个不小于  $x$  的数。

**输入格式:** 第一行  $n$   $q$ , 第二行数组, 之后  $q$  行询问。

**输出格式:** 每行输出答案, 不存在输出 NONE。

样例输入:

```
5 4
7 1 5 3 9
4
9
10
1
```

样例输出:

```
5
9
NONE
1
```

完整代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    sort(a.begin() + 1, a.end());
    while (q--) {
        int x;
        cin >> x;
        auto it = lower_bound(a.begin() + 1, a.end(), x);
        if (it == a.end()) cout << "NONE\n";
        else cout << *it << '\n';
    }
    return 0;
}
```

测试设计: 额外测试: 构造最小规模、重复值、边界值各一组, 和样例一起运行。

---